

■ 데이터베이스

- DBMS (DataBase Management System)
- 여러 사람들이 **공유**하고 사용할 목적으로 통합 관리되는 정보의 집합
- 파일 시스템의 문제를 해결하기 위해 제시된 소프트웨어
 - 파일 시스템의 문제점
 1. 실시간 공유
 2. 중복 데이터
 3. 보안 (랜덤웨어 등)
 4. 안전하지 못한 보관

※ 데이터베이스 시스템의 단점

1. 많은 비용
2. 전문 지식

■ 데이터베이스

● 주요 기능

- 정의 : 데이터를 저장하기 위한 구조를 정의하거나 수정
- 조작 : 데이터 삽입 / 조회 / 수정 / 삭제
- 제어 : 데이터를 항상 정확하고 안전하게 유지 (권한, 저장, 복구 등)

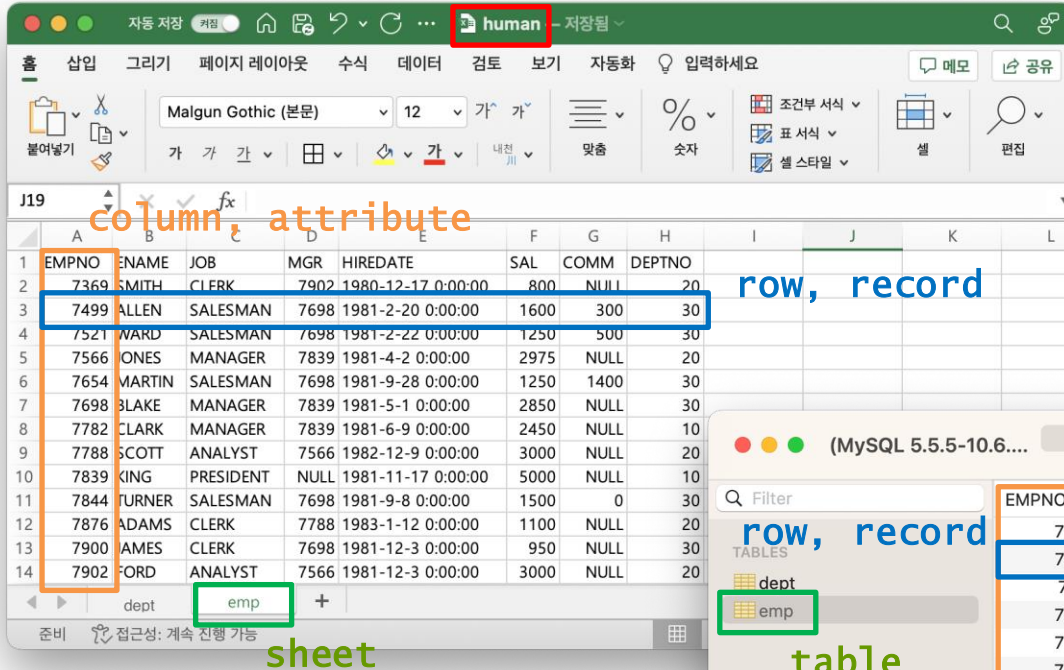
● SQL (Structured Query Language)

- 데이터베이스의 주요 기능을 실행하기 위한 언어
- 주요 기능을 실행하는 SQL
 - DDL (Data Definition Language) - 정의어
 - DML (Data Manipulation Language) - 조작어
 - DCL (Data Control Language) - 제어어

■ 엑셀 vs 데이터베이스

● 엑셀과 매우 유사한 형태

excel file

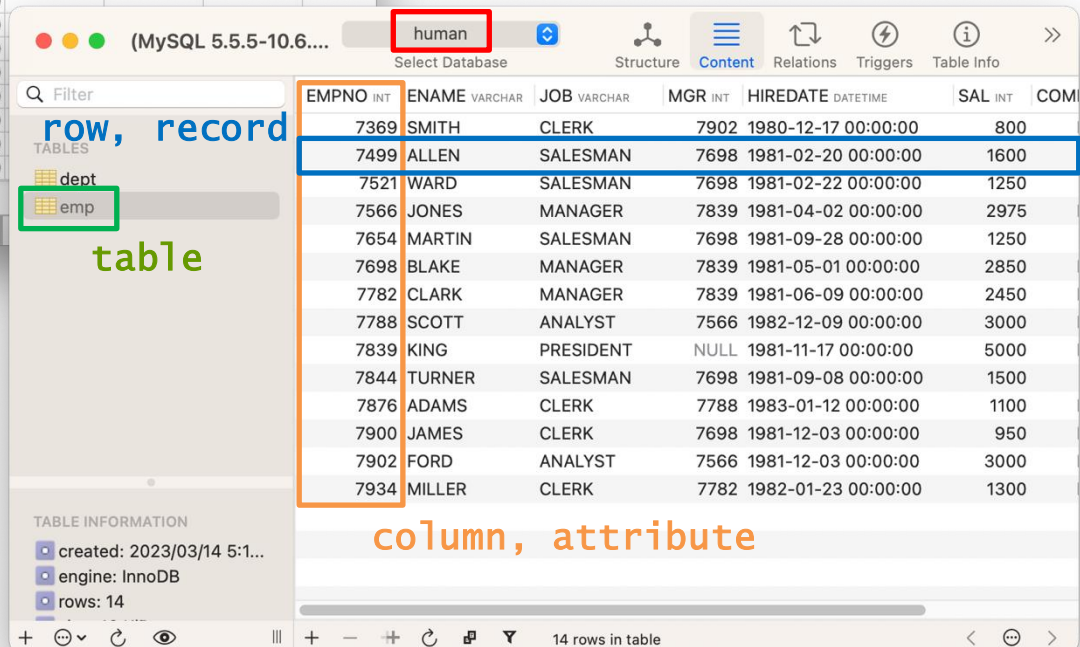


column, attribute

row, record

sheet

database



row, record

table

column, attribute

■ 엑셀 vs 데이터베이스

- 엑셀은 입력할 수 있는 여유 셀이 있으면 얼마든지 데이터 입력이 가능하지만 데이터베이스는 미리 구조를 정해놓고 데이터를 입력해야 됨
- 엑셀은 숫자 컬럼에 문자나 날짜 형식을 넣어도 되지만 데이터베이스는 입력 불가

■ Key

- 데이터를 정확히 찾아내기 위해서 고유한 값 필요 → **Key**
- Key의 종류
 - **Primary Key, PK (기본키) : 고유한 값 중 선택된 컬럼**
 - **Foreign Key, FK (외래키) : 다른 테이블의 기본키를 참조하는 컬럼**
 - Candidate Key (후보키) : 기본키로 선택될 수 있는 컬럼
 - Alternate Key (대체키) : 기본키로 선택되지 않은 후보키
 - Super Key (수퍼키) : 여러개의 컬럼을 묶어서 고유한 값으로 사용되는 컬럼의 집합



■ Key

● 기본키, 외래키가 사용된 모습

기본키

이름:	dept						
열:	+ 추가 × 제거 ▲ 위로 ▼ 아래로						
#	이름	데이터 유형	길이/설정	부호 없음	NULL 허...	0으로...	기본값
	1 DEPTNO	TINYINT	4	☐	☐	☐	기본값 없음
2	DNAME	VARCHAR	14	☐	☒	☐	NULL
3	LOC	VARCHAR	13	☐	☒	☐	NULL

기본키

이름:	emp						
열:	+ 추가 × 제거 ▲ 위로 ▼ 아래로						
#	이름	데이터 유형	길이/설정	부호 없음	NULL 허...	0으로...	기본값
	1 EMPNO	INT	11	☐	☐	☐	기본값 없음
2	ENAME	VARCHAR	10	☐	☒	☐	NULL
3	JOB	VARCHAR	9	☐	☒	☐	NULL
4	MGR	INT	11	☐	☒	☐	NULL
5	HIREDATE	DATETIME		☐	☒	☐	NULL
6	SAL	INT	11	☐	☒	☐	NULL
7	COMM	INT	11	☐	☒	☐	NULL
	8 DEPTNO	TINYINT	4	☐	☒	☐	NULL

외래키

■ 무결성 제약조건 (integrity constraint)

● 데이터의 무결성을 보장하고 일관된 상태로 유지하기 위한 규칙

※ 무결성 : 데이터를 정확하고 유효하게 유지하는 상태 (결함이 없는 상태)

1. 개체 무결성 제약조건 (PK)

	고객아이디	고객이름	나이	등급	직업	적립금
	apple	김현준	20	gold	학생	1000
사용불가	NULL	정소화	25	vip	간호사	2500
	carrot	원유선	28	gold	교사	4500
사용불가	NULL	정지영	22	silver	학생	0

기본키를 구성하는 값은 NULL 불가

2. 참조 무결성 제약조건 (FK)

고객 릴레이션

고객아이디	고객이름	나이	등급	직업	적립금
apple	김현준	20	gold	학생	1000
banana	정소화	25	vip	간호사	2500
carrot	원유선	28	gold	교사	4500
orange	정지영	22	silver	학생	0

주문 릴레이션

주문번호	주문고객	주문제품	수량	단가	주문일자
1001	cherry	신짜우동	10	2000	2013-01-01
1002	carrot	맛있는파이	5	500	2013-01-10
1003	banana	그대로만두	11	4500	2013-01-11

기본키로 존재하지 않는 값은
외래키로 사용 불가 (단, null은 가능)

■ 데이터 모델링

- 데이터를 데이터베이스로 저장할 수 있도록 설계하는 과정
- 설계된 결과를 표현하는 설계도 (ERD - Entity Relationship Model)

emp		사원			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
empno	사원번호	N/A	INT	N·N	
ename	사원명	N/A	VARCHAR(10)	NULL	
job	직무	N/A	VARCHAR(9)	NULL	
mgr	매니저	N/A	INT	NULL	
hiredate	입사일자	N/A	DATETIME	NULL	
sal	급여	N/A	INT	NULL	
comm	보너스	N/A	INT	NULL	
deptno	부서번호	N/A	TINYINT	NULL	

dept		부서			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
deptno	부서번호	N/A	TINYINT	N·N	
dname	부서명	N/A	VARCHAR(14)	NULL	
loc	부서위치	N/A	VARCHAR(13)	NULL	

warranty		보증서			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
model_nm	모델명	N/A	VARCHAR(50)	NULL	
period	보증기간	N/A	INT	NULL	

product		물품			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
name	물품명	N/A	VARCHAR(50)	NULL	
price	가격	N/A	INT	NULL	
animal_id	동물 아이디	N/A	INT	NULL	
warranty_id	보증서 아이디	N/A	INT	NULL	

owner		사람			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
name	이름	N/A	VARCHAR(50)	NULL	

animal		반려동물			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
name	이름	N/A	VARCHAR(50)	NULL	
age	나이	N/A	INT	NULL	
owner_id	사람 아이디	N/A	INT	NULL	

playground		놀이터			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
name	놀이터명	N/A	VARCHAR(50)	NULL	
address	주소	N/A	VARCHAR(50)	NULL	
tel	전화번호	N/A	VARCHAR(20)	NULL	

playground_animals		놀이터_반려동물			
물리 이름*	논리 이름*	도메인	데이터 타입	널 허용	
id	아이디	N/A	INT	N·N	
playground_id	놀이터 아이디	N/A	INT	NULL	
animal_id	동물 아이디	N/A	INT	NULL	

■ 데이터 정의어 (DDL)

● 기본 문법 구조

```
1 CREATE TABLE 테이블명 (  
2   컬럼명1 데이터타입 NOT NULL, // 필수 입력 컬럼 정의  
3   컬럼명2 데이터타입 DEFAULT 기본값, // 기본값이 있는 컬럼  
4   ...  
5   PRIMARY KEY (컬럼명), // 기본키 제약조건 (행 식별자)  
6   UNIQUE (컬럼명), // 생략가능  
7   FOREIGN KEY (컬럼명) REFERENCES 참조테이블 (컬럼명) // 생략가능  
8 );
```

■ 데이터 정의어 (DDL)

● 자주 사용되는 속성의 데이터 타입

종류	데이터 타입	의미
숫자	INT (INTEGER)	정수
	TINYINT, BIGINT, FLOAT, ...	정수 및 실수
문자	CHAR	고정 길이 문자열
	VARCHAR	가변 길이 문자열
	TEXT, LONGTEXT, ...	가변 길이 문자열
날짜	DATETIME	년, 월, 일로 표현되는 날짜
	DATE, TIMESTAMP, YEAR, ...	년, 월, 일로 표현되는 날짜
기타	ENUM, BOOL, ...	열거형, 논리, ...

■ 데이터 정의어 (DDL)

● '부서' 테이블 생성

#	이름	데이터 유형	길이/설정	부호 없음	NULL 허...
 1	부서번호	INT	11	☐	☐
2	부서이름	VARCHAR	10	☐	☒

```
CREATE TABLE 부서 (  
    부서번호 INT,  
    부서이름 VARCHAR(10),  
    PRIMARY KEY(부서번호)  
);
```

● '사원' 테이블 생성

#	이름	데이터 유형	길이/설정	부호 없음	NULL 허...
 1	사원번호	INT	11	☐	☐
2	사원이름	VARCHAR	10	☐	☒
 3	소속부서	INT	11	☐	☒

```
CREATE TABLE 사원 (  
    사원번호 INT,  
    사원이름 VARCHAR(10),  
    소속부서 INT,  
    PRIMARY KEY(사원번호),  
    FOREIGN KEY(소속부서) REFERENCES 부서(부서번호)  
);
```

■ 연습문제

● 아래 조건에 맞는 "고객" 테이블을 생성하는 SQL 작성하기

- 고객아이디 (문자, 최대 20자, 기본키)
- 이름 (문자, 최대 10자, NULL 불가)
- 나이 (정수)
- 등급 (문자, 최대 10자, 기본값 'normal')
- 직업 (문자, 최대 10자, 기본값 없음)
- 적립금 (정수, 기본값 0)

고객	
A-Z 고객아이디	varchar(20) NOT NULL
A-Z 이름	varchar(10) NOT NULL
123 나이	int(11)
A-Z 등급	varchar(10)
A-Z 직업	varchar(10)
123 적립금	int(11)

■ 연습문제

● 아래 조건에 맞는 "고객" 테이블을 생성하는 SQL 작성하기

- 고객아이디 (문자, 최대 20자, 기본키)
- 이름 (문자, 최대 10자, NULL 불가)
- 나이 (정수)
- 등급 (문자, 최대 10자, 기본값 'normal')
- 직업 (문자, 최대 10자, 기본값 없음)
- 적립금 (정수, 기본값 0)

```
CREATE TABLE 고객 (  
  고객아이디 VARCHAR(20) PRIMARY KEY,  
  이름 VARCHAR(10) NOT NULL,  
  나이 INT,  
  등급 VARCHAR(10) DEFAULT 'normal',  
  직업 VARCHAR(10) DEFAULT '',  
  적립금 INT DEFAULT 0  
);
```

■ 연습문제

● "제품" 테이블에 "제조업체" 컬럼 추가하기

```
CREATE TABLE 제품 (  
    제품번호 CHAR(3) PRIMARY KEY,  
    제품명 VARCHAR(20),  
    재고량 INT, 단가 INT  
);
```

– ALTER TABLE 사용

(데이터 타입은 VARCHAR(20)이며, NULL 허용)

제품	
A-Z 제품번호	char(3) NOT NULL
A-Z 제품명	varchar(20)
123 재고량	int(11)
123 단가	int(11)



제품	
A-Z 제품번호	char(3) NOT NULL
A-Z 제품명	varchar(20)
123 재고량	int(11)
123 단가	int(11)
A-Z 제조업체	varchar(20)

■ 연습문제

● "제품" 테이블에 "제조업체" 컬럼 추가하기

```
CREATE TABLE 제품 (  
    제품번호 CHAR(3) PRIMARY KEY,  
    제품명 VARCHAR(20),  
    재고량 INT, 단가 INT  
);
```

– ALTER TABLE 사용

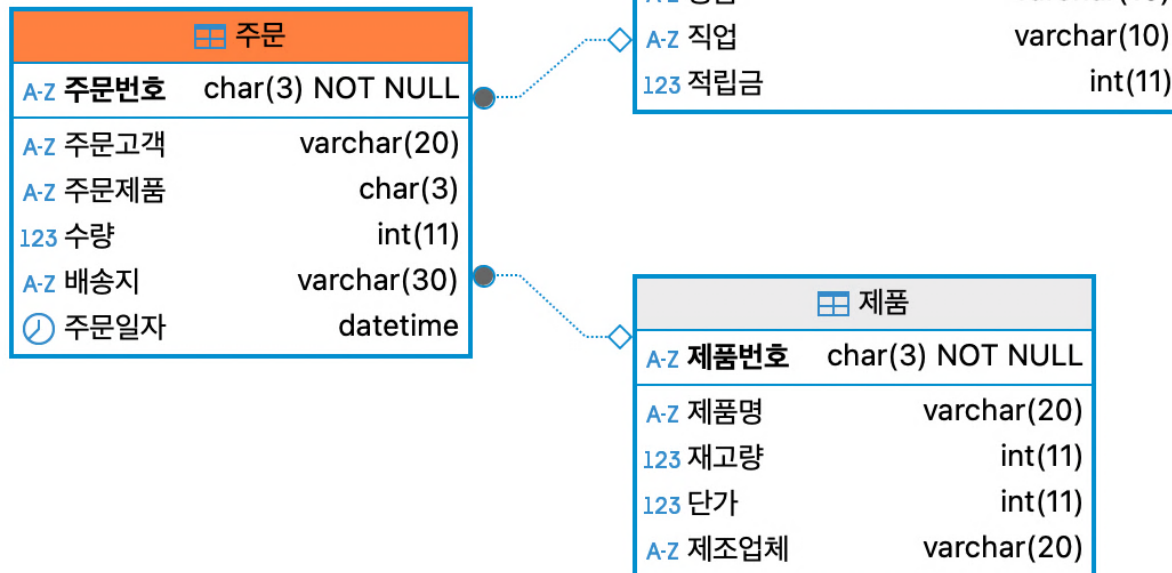
(데이터 타입은 VARCHAR(20)이며, NULL 허용)

```
ALTER TABLE 제품 ADD 제조업체 VARCHAR(20);
```

■ 연습문제

● 아래 조건에 맞는 "주문" 테이블을 생성하는 SQL 작성하기

- 주문번호 (문자 3자, 기본키)
- 주문고객 (문자 20자) : 고객 테이블의 기본키 참조, 삭제 시 함께 삭제
- 주문제품 (문자 3자) : 제품 테이블의 기본키 참조, 수정 시 함께 수정
- 수량 (정수)
- 배송지 (문자 30자)
- 주문일자 (DATETIME)



■ 연습문제

● 아래 조건에 맞는 "주문" 테이블을 생성하는 SQL 작성하기

- 주문번호 (문자 3자, 기본키)
- 주문고객 (문자 20자) : 고객 테이블의 기본키 참조, 삭제 시 함께 삭제
- 주문제품 (문자 3자) : 제품 테이블의 기본키 참조, 수정 시 함께 수정
- 수량 (정수)
- 배송지 (문자 30자)
- 주문일자 (DATETIME)

```
CREATE TABLE 주문 (  
    주문번호 CHAR(3), 주문고객 VARCHAR(20),  
    주문제품 CHAR(3), 수량 INT,  
    배송지 VARCHAR(30), 주문일자 DATETIME,  
    FOREIGN KEY (주문고객) REFERENCES 고객(고객아이디)  
        ON DELETE CASCADE,  
    FOREIGN KEY (주문제품) REFERENCES 제품(제품번호)  
        ON UPDATE CASCADE,  
    PRIMARY KEY (주문번호)  
);
```

■ 데이터 삽입 : INSERT

```
INSERT INTO 테이블명  
    [(속성 리스트)]  
    VALUES  
    (속성의 값 리스트);
```

- 속성 리스트를 생략하면 테이블을 정의할 때 지정한 속성의 순서대로 값 삽입
- VALUES 와 함께 삽입할 속성의 값을 입력
- INTO 의 속성과 VALUES 속성은 순서대로 1:1 대응되어야 됨

■ 데이터 삽입 : INSERT

● '부서' 테이블에 데이터 삽입

부서번호 INT	부서이름 VARCHAR
1	인사부
2	연구부
3	홍보부

```
INSERT INTO 부서 (부서번호, 부서이름) VALUES (1, '인사부');  
INSERT INTO 부서 (부서번호, 부서이름) VALUES (2, '연구부');  
INSERT INTO 부서 (부서번호, 부서이름) VALUES (3, '홍보부');
```

● '사원' 테이블에 데이터 삽입

사원번호 INT	사원이름 VARCHAR	소속부서 INT
1001	AAA	3 →
1002	BBB	2 →
1003	CCC	1 →

```
INSERT INTO 사원 (사원번호, 사원이름, 소속부서) VALUES (1001, 'AAA', 3);  
INSERT INTO 사원 (사원번호, 사원이름, 소속부서) VALUES (1002, 'BBB', 2);  
INSERT INTO 사원 (사원번호, 사원이름, 소속부서) VALUES (1003, 'CCC', 1);
```

■ 데이터 삽입 : INSERT

● '고객' 테이블에 데이터 삽입

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
apple	정소화	20	gold	학생	1000
banana	김선우	25	vip	간호사	2500
carrot	고명석	28	gold	교사	4500
melon	성원용	35	gold	회사원	5000
orange	김용욱	22	silver	학생	0
peach	오형준	NULL	silver	의사	300
pear	채광주	31	silver	회사원	500

```
INSERT INTO 고객 (고객아이디, 이름, 나이, 등급, 직업, 적립금)
VALUES
('apple', '정소화', 20, 'gold', '학생', 1000),
('banana', '김선우', 25, 'vip', '간호사', 2500),
('carrot', '고명석', 28, 'gold', '교사', 4500),
('orange', '김용욱', 22, 'silver', '학생', 0),
('melon', '성원용', 35, 'gold', '회사원', 5000),
('peach', '오형준', NULL, 'silver', '의사', 300),
('pear', '채광주', 31, 'silver', '회사원', 500);
```

■ 데이터 삽입 : INSERT

● '제품' 테이블에 데이터 삽입

제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운짬면	2500	5500	민국푸드
p03	콩떡파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과

```
INSERT INTO 제품 (제품번호, 제품명, 재고량, 단가, 제조업체)
VALUES
('p01', '그냥만두', 5000, 4500, '대한식품'),
('p02', '매운짬면', 2500, 5500, '민국푸드'),
('p03', '콩떡파이', 3600, 2600, '한빛제과'),
('p04', '맛난초콜렛', 1250, 2500, '한빛제과'),
('p05', '얼큰라면', 2200, 1200, '대한식품'),
('p06', '통통우동', 1000, 1550, '민국푸드'),
('p07', '달콤비스킷', 1650, 1500, '한빛제과');
```

■ 데이터 삽입 : INSERT

● '주문' 테이블에 데이터 삽입

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o01	apple	➔ p03	➔ 10	서울시 마포구	2013-01-01 00:00:00
o02	melon	➔ p01	➔ 5	인천시 계양구	2013-01-10 00:00:00
o03	banana	➔ p06	➔ 45	경기도 부천시	2013-01-11 00:00:00
o04	carrot	➔ p02	➔ 8	부산시 금정구	2013-02-01 00:00:00
o05	melon	➔ p06	➔ 36	경기도 용인시	2013-02-20 00:00:00
o06	banana	➔ p01	➔ 19	충청북도 보은군	2013-03-02 00:00:00
o07	apple	➔ p03	➔ 22	서울시 영등포구	2013-03-15 00:00:00
o08	pear	➔ p02	➔ 50	강원도 춘천시	2013-04-10 00:00:00
o09	banana	➔ p04	➔ 15	전라남도 목포시	2013-04-11 00:00:00
o10	carrot	➔ p03	➔ 20	경기도 안양시	2013-05-22 00:00:00

INSERT INTO 주문 (주문번호, 주문고객, 주문제품, 수량, 배송지, 주문일자)
VALUES

('o01', 'apple', 'p03', 10, '서울시 마포구', '2013-01-01'),
('o02', 'melon', 'p01', 5, '인천시 계양구', '2013-01-10'),
('o03', 'banana', 'p06', 45, '경기도 부천시', '2013-01-11'),
('o04', 'carrot', 'p02', 8, '부산시 금정구', '2013-02-01'),
('o05', 'melon', 'p06', 36, '경기도 용인시', '2013-02-20'),
('o06', 'banana', 'p01', 19, '충청북도 보은군', '2013-03-02'),
('o07', 'apple', 'p03', 22, '서울시 영등포구', '2013-03-15'),
('o08', 'pear', 'p02', 50, '강원도 춘천시', '2013-04-10'),
('o09', 'banana', 'p04', 15, '전라남도 목포시', '2013-04-11'),
('o10', 'carrot', 'p03', 20, '경기도 안양시', '2013-05-22');

■ 연습문제

● "부서" 테이블에 데이터를 삽입하기

- 부서번호 4, 부서이름 '총무부'
- 부서번호 5, 부서이름 '개발부'

부서번호 INT	부서이름 VARCHAR
4	총무부
5	개발부

● 속성 리스트를 생략하여 "사원" 테이블에 데이터를 삽입하는 SQL 작성하기

- 사원번호 1004, 사원이름 'DDD', 소속부서 2

사원번호 INT	사원이름 VARCHAR	소속부서 INT
1004	DDD	2

● 한 번의 INSERT 문으로 "고객" 테이블에 아래 데이터를 삽입하는 SQL 작성하기

- 고객아이디 'kiwi', 이름 '박서준', 나이 29, 등급 'gold', 직업 '배우', 적립금 4000
- 고객아이디 'plum', 이름 '이하늘', 나이 40, 등급 'vip', 직업 '가수', 적립금 8000

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
kiwi	박서준	29	gold	배우	4000
plum	이하늘	40	vip	가수	8000

■ 연습문제

● "부서" 테이블에 데이터를 삽입하기

```
INSERT INTO 부서 (부서번호, 부서이름)
VALUES (4, '총무부'), (5, '개발부');
```

● 속성 리스트를 생략하여 "사원" 테이블에 데이터를 삽입하는 SQL 작성하기

```
INSERT INTO 사원
VALUES (1004, 'DDD', 2);
```

● 한 번의 INSERT 문으로 "고객" 테이블에 아래 데이터를 삽입하는 SQL 작성하기

```
INSERT INTO 고객 (고객아이디, 이름, 나이, 등급, 직업, 적립금)
VALUES ('kiwi', '박서준', 29, 'gold', '배우', 4000),
      ('plum', '이하늘', 40, 'vip', '가수', 8000);
```


■ 데이터 조회 : SELECT

```
SELECT [DISTINCT] 속성 리스트  
FROM 테이블 리스트
```

- 검색하려는 속성의 이름 나열
- 검색하려는 속성이 있는 테이블의 이름 나열
- 검색 결과는 테이블 형태로 반환
- DISTINCT 사용 시 검색 결과 중 중복 튜플은 하나만 출력

■ 데이터 조회 : SELECT

- '고객' 테이블에서 고객아이디, 이름, 등급 속성 검색

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR
apple	정소화	gold
banana	김선우	vip
carrot	고명석	gold
kiwi	박서준	gold
melon	성원용	gold
orange	김용욱	silver
peach	오형준	silver
pear	채광주	silver
plum	이하늘	vip

```
SELECT 고객아이디, 이름, 등급  
FROM 고객;
```

■ 데이터 조회 : SELECT

● '고객' 테이블의 모든 속성 검색

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
apple	정소화	20	gold	학생	1000
banana	김선우	25	vip	간호사	2500
carrot	고명석	28	gold	교사	4500
kiwi	박서준	29	gold	배우	4000
melon	성원용	35	gold	회사원	5000
orange	김용욱	22	silver	학생	0
peach	오형준	NULL	silver	의사	300
pear	채광주	31	silver	회사원	500
plum	이하늘	40	vip	가수	8000

```
SELECT 고객아이디, 이름, 나이, 등급, 직업, 적립금  
FROM 고객;
```

```
SELECT *  
FROM 고객;
```

■ 데이터 조회 : SELECT

● '제품' 테이블에서 제조업체 검색

제조업체 VARCHAR
대한식품
민국푸드
한빛제과
한빛제과
대한식품
민국푸드
한빛제과

```
SELECT 제조업체  
FROM 제품;
```

● '제품' 테이블에서 제조업체 중복없이 검색

제조업체 VARCHAR
대한식품
민국푸드
한빛제과

```
SELECT DISTINCT 제조업체  
FROM 제품;
```

■ 데이터 조회 : SELECT

● 속성명 변경 출력 : AS

```
SELECT 속성명 [AS] 변경속성  
FROM 테이블
```

- 테이블이 가지고 있는 기존 속성명을 지정한 속성명으로 변경하여 출력
- 변경하는 속성명에 공백이 포함되어 있으면 따옴표로 묶어줌
- AS 키워드 생략 가능

■ 데이터 조회 : SELECT

- '제품' 테이블에서 제품명과 단가를 검색, 단가를 가격으로 바꾸어 출력

제품명 VARCHAR	가격 INT
그냥만두	4500
매운짬면	5500
콩떡파이	2600
맛난초콜렛	2500
얼큰라면	1200
통통우동	1550
달콤비스킷	1500

```
SELECT 제품명, 단가 AS '가격'  
FROM 제품;
```

■ 데이터 조회 : SELECT

● 산술식을 이용한 검색

```
SELECT 속성명 + 숫자, 속성명 * 숫자, ...  
FROM 테이블
```

- 속성명과 +, -, *, / 등의 산술 연산자와 상수로 구성
- 실제 속성의 값은 변하지 않고 결과 테이블에서만 계산된 값이 출력

■ 데이터 조회 : SELECT

- '제품' 테이블에서 제품명과 단가를 검색,
단가에 500원을 더한 후 조정단가라는 속성명으로 바꾸어 출력

제품명 VARCHAR	조정단가 BIGINT
그냥만두	5000
매운짬면	6000
콩떡파이	3100
맛난초콜렛	3000
얼큰라면	1700
통통우동	2050
달콤비스킷	2000

```
SELECT 제품명, 단가 + 500 AS '조정 단가'  
FROM 제품;
```


■ 데이터 조회 : SELECT

● 조건에 해당하는 데이터 검색 : WHERE

```
SELECT [DISTINCT] 속성 리스트  
FROM 테이블 리스트  
[WHERE 조건];
```

- 비교 / 논리 연산자 등을 이용하여 검색 조건 작성
- 숫자, 문자, 날짜 모두 조건으로 사용 가능
- 문자 또는 날짜 값 사용시 따옴표로 묶어서 표현

종류	연산자	의미	연산자	의미	연산자	의미
비교	=	같다	<	작다	<=	작거나 같다
	<> 또는 !=	다르다	>	크다	>=	크거나 같다
논리	AND	모두 참	OR	1개 이상 참	NOT	거짓일때 참

■ 데이터 조회 : SELECT

- '제품' 테이블에서 한빛제과가 제조한 제품의 제품명, 재고량, 단가 검색

제품명 VARCHAR	재고량 INT	단가 INT
쿵떡파이	3600	2600
맛난초콜렛	1250	2500
달콤비스킷	1650	1500

```
SELECT 제품명, 재고량, 단가  
FROM 제품  
WHERE 제조업체 = '한빛제과';
```

■ 데이터 조회 : SELECT

- '주문' 테이블에서 apple 고객아이디로 15개 이상 주문한
주문제품, 수량, 주문일자 검색

주문제품 CHAR	수량 INT	주문일자 DATETIME
-----------	--------	---------------

p03	22	2013-03-15 00:00:00
-----	----	---------------------

```
SELECT 주문 제품, 수량, 주문 일자  
FROM 주문  
WHERE 주문 고객 = 'apple' AND 수량 >= 15;
```

■ 데이터 조회 : SELECT

- '주문' 테이블에서 apple 고객아이디로 주문했거나 15개 이상 주문된 제품의 주문제품, 수량, 주문일자, 주문고객 검색

주문제품 CHAR	수량 INT	주문일자 DATETIME
p03	10	2013-01-01 00:00:00
p06	45	2013-01-11 00:00:00
p06	36	2013-02-20 00:00:00
p01	19	2013-03-02 00:00:00
p03	22	2013-03-15 00:00:00
p02	50	2013-04-10 00:00:00
p04	15	2013-04-11 00:00:00
p03	20	2013-05-22 00:00:00

```
SELECT 주문 제품, 수량, 주문 일자  
FROM 주문  
WHERE 주문 고객 = 'apple' OR 수량 >= 15;
```

■ 데이터 조회 : SELECT

- '제품' 테이블에서 단가가 2000원 이상이고 3000원 이하 인 제품의 제품명, 단가, 제조업체 검색

제품명 VARCHAR	단가 INT	제조업체 VARCHAR
쿵떡파이	2600	한빛제과
맛난초콜렛	2500	한빛제과

```
SELECT 제품명, 단가, 제조업체  
FROM 제품  
WHERE 단가 >= 2000 AND 단가 <= 3000;
```

```
SELECT 제품명, 단가, 제조업체  
FROM 제품  
WHERE 단가 BETWEEN 2000 AND 3000;
```

■ 연습문제

- 마케팅팀에서 골드 등급 고객에게만 혜택을 제공하려고 합니다.
등급이 'gold'인 고객의 이름과 적립금을 조회하는 SQL을 작성하세요.

이름 VARCHAR	적립금 INT
정소화	1000
고명석	4500
박서준	4000
성원용	5000

- 창고팀에서 고가 상품을 조사하려고 합니다.
단가가 4000원 이상인 제품명과 단가를 조회하세요.

제품명 VARCHAR	단가 INT
그냥만두	4500
매운짬면	5500

- 재고가 2000개 이하인 상품을 파악하려고 합니다.
제품명과 재고량을 조회하세요.

제품명 VARCHAR	재고량 INT
맛난초콜렛	1250
통통우동	1000
달콤비스킷	1650

■ 연습문제

- 마케팅팀에서 골드 등급 고객에게만 혜택을 제공하려고 합니다.
등급이 'gold'인 고객의 이름과 적립금을 조회하는 SQL을 작성하세요.

```
SELECT 이름, 적립금  
FROM 고객  
WHERE 등급 = 'gold';
```

- 창고팀에서 고가 상품을 조사하려고 합니다.
단가가 4000원 이상인 제품명과 단가를 조회하세요.

```
SELECT 제품명, 단가  
FROM 제품  
WHERE 단가 >= 4000;
```

- 재고가 2000개 이하인 상품을 파악하려고 합니다.
제품명과 재고량을 조회하세요.

```
SELECT 제품명, 재고량  
FROM 제품  
WHERE 재고량 <= 2000;
```

■ 데이터 조회 : SELECT

● 부분적으로 일치하는 데이터 검색 : LIKE

```
SELECT 속성 리스트  
FROM 테이블  
WHERE 속성명 LIKE '검색 데이터';
```

- 문자열을 이용하는 조건에만 적용 가능

기호	설명
%	0개 이상의 문자 (문자 내용과 개수 상관없이 모두 검색)
_ (underscore)	_ 개수 만큼의 문자 (문자 내용과 상관없이 모두 검색)

사용 예	설명
LIKE '데이터%'	데이터로 시작하는 문자열
LIKE '%데이터'	데이터로 끝나는 문자열
LIKE '%데이터%'	데이터가 포함된 문자열
LIKE '데이터__'	데이터로 시작하는 6자 길이의 문자열
LIKE '__한%'	세 번째 글자가 '한'인 문자열

■ 데이터 조회 : SELECT

- '고객' 테이블에서 성이 '김'인 고객의 고객이름, 나이, 등급, 적립금 검색

이름 VARCHAR	나이 INT	등급 VARCHAR	적립금 INT
김선우	25	vip	2500
김용욱	22	silver	0

```
SELECT 이름, 나이, 등급, 적립금  
FROM 고객  
WHERE 이름 LIKE '김%';
```

■ 데이터 조회 : SELECT

- '고객' 테이블에서 고객아이디가 5자인 고객의 고객아이디, 고객이름, 등급 검색

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR
apple	정소화	gold
melon	성원용	gold
peach	오형준	silver

```
SELECT 고객아이디, 고객이름, 등급  
FROM 고객  
WHERE 고객아이디 LIKE '_____';
```

■ 데이터 조회 : SELECT

● NULL 데이터 검색 : IS NULL / IS NOT NULL

```
SELECT 속성 리스트  
FROM 테이블  
WHERE 속성명 IS NULL;
```

- 특정 속성의 값이 NULL 값인지 비교
- IS 키워드를 사용하지 않고 비교연산자 이용 시 항상 거짓으로 출력

■ 데이터 조회 : SELECT

- '고객' 테이블에서 나이가 입력되지 않은 (NULL) 고객의 고객이름 검색

이름 VARCHAR
오형준

```
SELECT 이름  
FROM 고객  
WHERE 나이 IS NULL;
```

- '고객' 테이블에서 나이가 입력되어 있는 고객의 고객이름 검색

이름 VARCHAR
정소화
김선우
고명석
박서준
성원용
김용욱
채광주
이하늘

```
SELECT 이름  
FROM 고객  
WHERE 나이 IS NOT NULL;
```

■ 연습문제

- 마케팅팀에서 이름에 '석'이 들어가는 고객 목록을 확인하려고 합니다.

이름, 나이, 등급을 조회하세요.

이름 VARCHAR	나이 INT	등급 VARCHAR
고명석	28	gold

- 고객아이디가 정확히 4글자인 고객을 찾으세요.

고객아이디, 이름, 등급을 조회하세요.

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR
kiwi	박서준	gold
pear	채광주	silver
plum	이하늘	vip

■ 연습문제

- 마케팅팀에서 이름에 '석'이 들어가는 고객 목록을 확인하려고 합니다.

이름, 나이, 등급을 조회하세요.

```
SELECT 이름, 나이, 등급  
FROM 고객  
WHERE 이름 LIKE '%석%';
```

- 고객아이디가 정확히 4글자인 고객을 찾으세요.

고객아이디, 이름, 등급을 조회하세요.

```
SELECT 고객아이디, 이름, 등급  
FROM 고객  
WHERE 고객아이디 LIKE '____';
```

■ 데이터 조회 : SELECT

● 데이터 검색 후 정렬 : ORDER BY

```
SELECT [DISTINCT] 속성 리스트  
FROM 테이블  
[WHERE 조건]  
[ORDER BY 속성 리스트 [ASC | DESC]];
```

- 검색 결과를 오름 / 내림차순으로 정렬하여 출력
- 오름차순 : ASC, 내림차순 : DESC
- 속성명을 여러 개 사용하는 경우 왼쪽 속성을 우선으로 정렬

■ 데이터 조회 : SELECT

- '고객' 테이블에서 고객이름, 등급, 나이를 나이의 내림차순으로 검색

이름 VARCHAR	등급 VARCHAR	나이 INT
이하늘	vip	40
성원용	gold	35
채광주	silver	31
박서준	gold	29
고명석	gold	28
김선우	vip	25
김용욱	silver	22
정소화	gold	20
오형준	silver	NULL

```
SELECT 이름, 등급, 나이  
FROM 고객  
ORDER BY 나이 DESC;
```


■ 데이터 조회 : SELECT

- '주문' 테이블에서 수량이 10개 이상인 주문의 주문고객, 수량, 주문일자를 검색
단, 주문제품을 기준으로 오름차순 정렬하고 주문제품이 같은 경우
수량을 기준으로 내림차순 정렬

주문고객 VARCHAR	주문제품 CHAR	수량 INT	주문일자 DATETIME
banana	p01	19	2013-03-02 00:00:00
pear	p02	50	2013-04-10 00:00:00
apple	p03	22	2013-03-15 00:00:00
carrot	p03	20	2013-05-22 00:00:00
apple	p03	10	2013-01-01 00:00:00
banana	p04	15	2013-04-11 00:00:00
banana	p06	45	2013-01-11 00:00:00
melon	p06	36	2013-02-20 00:00:00

```
SELECT 주문고객, 주문제품, 수량, 주문일자  
FROM 주문  
WHERE 수량 >= 10  
ORDER BY 주문제품 ASC, 수량 DESC;
```

■ 연습문제

- 영업팀에서 2013년 3월에 주문한 내역을 날짜순으로 보고 싶어 합니다.

주문 테이블에서 주문번호, 주문고객, 주문일자를 조회하되

주문일자 오름차순으로 정렬하세요.

주문번호 CHAR	주문고객 VARCHAR	주문일자 DATETIME
o06	banana	2013-03-02 00:00:00
o07	apple	2013-03-15 00:00:00

- 배송팀에서 '서울시'로 시작하는 배송지 주문을 확인합니다.

주문 테이블에서 주문번호, 주문고객, 배송지를 조회하되,

주문고객 오름차순으로 정렬하세요.

주문번호 CHAR	주문고객 VARCHAR	배송지 VARCHAR
o01	apple	서울시 마포구
o07	apple	서울시 영등포구

- 창고팀에서 재고량이 2000개 이하인 제품을 확인하려고 합니다.

제품 테이블에서 제품명, 재고량, 단가를 조회하되,

재고량 오름차순, 재고량이 같으면 단가 내림차순으로 정렬하세요.

제품명 VARCHAR	재고량 INT	단가 INT
통통우동	1000	1550
맛난초콜렛	1250	2500
달콤비스킷	1650	1500

■ 연습문제

- 영업팀에서 2013년 3월에 주문한 내역을 날짜순으로 보고 싶어 합니다.
주문일자 오름차순으로 정렬하세요.

```
SELECT 주문번호, 주문고객, 주문일자  
FROM 주문  
WHERE 주문일자 BETWEEN '2013-03-01' AND '2013-03-31'  
ORDER BY 주문일자 ASC;
```

- 배송팀에서 '서울시'로 시작하는 배송지 주문을 확인합니다.
주문고객 오름차순으로 정렬하세요.

```
SELECT 주문번호, 주문고객, 배송지  
FROM 주문  
WHERE 배송지 LIKE '서울시%'  
ORDER BY 주문고객 ASC;
```

- 창고팀에서 재고량이 2000개 이하인 제품을 확인하려고 합니다.
재고량 오름차순, 재고량이 같으면 단가 내림차순으로 정렬하세요.

```
SELECT 제품명, 재고량, 단가  
FROM 제품  
WHERE 재고량 <= 2000  
ORDER BY 재고량 ASC, 단가 DESC;
```

■ 데이터 조회 : SELECT

● 검색 결과 제한 : LIMIT

```
SELECT [DISTINCT] 속성 리스트  
FROM 테이블  
[WHERE 조건]  
[ORDER BY 속성 리스트 [ASC | DESC]]  
LIMIT 시작위치, 개수;
```

- 검색 결과에서 출력할 행의 개수를 제한하여 표시
- LIMIT 개수 : 첫 번째 행부터 지정한 개수만큼 출력
- LIMIT 시작위치, 개수 : 시작위치는 0부터 시작하며
지정한 위치부터 개수만큼 출력
- 주로 페이지별 데이터 출력, 샘플 데이터 조회 등에 사용
- ORDER BY와 함께 쓰면 원하는 순서로 정렬된 일부 결과만 가져올 수 있음

■ 데이터 조회 : SELECT

- '고객' 테이블에서 적립금이 많은 순으로 상위 3명 조회

이름 VARCHAR	적립금 INT
이하늘	8000
성원용	5000
고명석	4500

```
SELECT 이름, 적립금  
FROM 고객  
ORDER BY 적립금 DESC  
LIMIT 3;
```

- '제품' 테이블에서 단가가 높은 순으로 정렬한 후 네번째 행부터 4개 제품 조회

제품명 VARCHAR	단가 INT
맛난초콜렛	2500
통통우동	1550
달콤비스킷	1500
얼큰라면	1200

```
SELECT 제품명, 단가  
FROM 제품  
ORDER BY 단가 DESC  
LIMIT 3, 4;
```

■ 연습문제

- 창고팀에서 가격이 높은 상품 중 상위 2개만 확인하려고 합니다.

제품 테이블에서 제품명, 단가를 단가 내림차순으로 정렬하여
상위 2개만 조회하세요.

제품명 VARCHAR	단가 INT
매운짬면	5500
그냥만두	4500

- 마케팅팀에서 적립금이 많은 고객 순으로 5번째부터 3명만 추출하려고 합니다.

고객 테이블에서 이름, 적립금을 적립금 내림차순으로 정렬하여,
5번째부터 3명만 조회하세요.

이름 VARCHAR	적립금 INT
김선우	2500
정소화	1000
채광주	500

■ 연습문제

- 창고팀에서 가격이 높은 상품 중 상위 2개만 확인하려고 합니다.

제품 테이블에서 제품명, 단가를 단가 내림차순으로 정렬하여
상위 2개만 조회하세요.

```
SELECT 제품명, 단가  
FROM 제품  
ORDER BY 단가 DESC  
LIMIT 2;
```

- 마케팅팀에서 적립금이 많은 고객 순으로 5번째부터 3명만 추출하려고 합니다.

고객 테이블에서 이름, 적립금을 적립금 내림차순으로 정렬하여,
5번째부터 3명만 조회하세요.

```
SELECT 이름, 적립금  
FROM 고객  
ORDER BY 적립금 DESC  
LIMIT 4, 3;
```

■ 데이터 조회 : SELECT

● 함수 사용

종류	함수	설명
숫자	ROUND(값, 반올림 지점)	반올림
	TRUNCATE(값, 버림 지점)	버림
	MOD(값-피제수, 나눌 값-제수)	나머지
문자	UPPER(문자열) / LOWER(문자열)	대/소문자로 변환
	LENGTH(문자열)	문자열 길이
	INSTR(대상 문자열, 찾을 문자열)	문자열 위치
	SUBSTR(대상 문자열, 시작 위치[, 추출 개수])	문자열 추출
	CONCAT(문자열1, 문자열2)	문자열 합치기
	LPAD(출력 문자열, 출력 자리수, 채워넣을 문자)	자리수 맞춰 문자 채워넣기
	RPAD(출력 문자열, 출력 자리수, 채워넣을 문자)	자리수 맞춰 문자 채워넣기
날짜	SYSDATE(), NOW()	현재 일시
	DATE_FORMAT(날짜, 출력형식)	날짜 형태의 데이터를 지정하는 형식에 맞춰서 출력

■ 데이터 조회 : SELECT

● ROUND

결과 #1 (1×1)
ROUND(10.123, 2)
10.12

```
SELECT ROUND(10.123, 2);
```

결과 #1 (1×1)
ROUND(3100.5, 0)
3,101

```
SELECT ROUND(3100.5, 0);
```

결과 #1 (1×1)
ROUND(127.5, -1)
130

```
SELECT ROUND(127.5, -1);
```

● TRUNCATE

결과 #1 (1×1)
TRUNCATE(10.123, 2)
10.12

```
SELECT TRUNCATE(10.123, 2);
```

결과 #1 (1×1)
TRUNCATE(3100.5, 0)
3,100

```
SELECT TRUNCATE(3100.5, 0);
```

● MOD

결과 #1 (1×1)
MOD(5, 2)
1

```
SELECT MOD(5, 2);
```

결과 #1 (1×1)
MOD(2, 2)
0

```
SELECT MOD(5, 2);
```

■ 데이터 조회 : SELECT

● UPPER / LOWER

결과 #1 (1×1)

UPPER('aa')
AA

```
SELECT UPPER('aa');
```

결과 #1 (1×1)

LOWER('AA')
aa

```
SELECT LOWER('AA');
```

● LENGTH

결과 #1 (1×1)

LENGTH('ABC123')
6

```
SELECT LENGTH('ABC123');
```

결과 #1 (1×1)

LENGTH('동해물과 백두산이')
25

```
SELECT LENGTH('동해물과 백두산이');
```

● INSTR

결과 #1 (1×1)

INSTR('HELLO DB', ' ')
6

```
SELECT INSTR('HELLO DB', ' ');
```

결과 #1 (1×1)

INSTR('HELLO JAVA', 'Z')
0

```
SELECT INSTR('HELLO JAVA', 'Z');
```

■ 데이터 조회 : SELECT

● SUBSTR

결과 #1 (1×1)
SUBSTR('HELLO DB', 7)
DB

```
SELECT SUBSTR('HELLO DB', 7);
```

결과 #1 (1×1)
SUBSTR('HELLO JAVA', 7, 2)
JA

```
SELECT SUBSTR('HELLO JAVA', 7, 2);
```

● CONCAT

결과 #1 (1×1)
CONCAT(10, '개')
10개

```
SELECT CONCAT(10, '개');
```

결과 #1 (1×1)
CONCAT('HELLO', ' ', 'PYTHON')
HELLO PYTHON

```
SELECT CONCAT('HELLO', ' ', 'PYTHON');
```

● LPAD / RPAD

결과 #1 (1×1)
LPAD('123', 5, '@')
@@123

```
SELECT LPAD('123', 5, '@');
```

결과 #1 (1×1)
RPAD('DATABASE', 10, '12345')
DATABASE12

```
SELECT RPAD('DATABASE', 10, '12345');
```

■ 데이터 조회 : SELECT

● NOW() / SYSDATE()

NOW() DATETIME

2025-08-14 02:43:08

SYSDATE() DATETIME

2025-08-14 02:43:57

```
SELECT NOW();
```

```
SELECT SYSDATE();
```

● DATE_FORMAT

DATE_FORMAT(NOW(), '%Y년 %m월 %d일 %H:%i:%s') VARCHAR

2025년 08월 14일 02:44:39

```
SELECT DATE_FORMAT(NOW(), '%Y년 %m월 %d일 %H:%i:%s');
```

DATE_FORMAT('2025/11/11 11:12:13', '%Y년 %m월 %d일 %H:%i:%s') VARCHAR

2025년 11월 11일 11:12:13

```
SELECT DATE_FORMAT('2025/11/11 11:12:13', '%Y년 %m월 %d일 %H:%i:%s');
```

■ 데이터 조회 : SELECT

● 조건문 사용 : IF

```
SELECT 제품명  
      , IF(단가 > 3000, '고가', '저가') AS 가격분류  
FROM 제품;
```

제품명 VARCHAR	가격분류 VARCHAR
그냥만두	고가
매운짬면	고가
콩떡파이	저가
맛난초콜렛	저가
얼큰라면	저가
통통우동	저가
달콤비스켓	저가

```
SELECT 고객아이디, 이름, 등급, 적립금,  
      IF(등급 = 'vip', 적립금 + 5000, 적립금) AS '보너스적립금'  
FROM 고객;
```

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR	적립금 INT	보너스적립금 BIGINT
apple	정소화	gold	1000	1000
banana	김선우	vip	2500	7500
carrot	고명석	gold	4500	4500
kiwi	박서준	gold	4000	4000
melon	성원용	gold	5000	5000
orange	김용욱	silver	0	0
peach	오형준	silver	300	300
pear	채광주	silver	500	500
plum	이하늘	vip	8000	13000

■ 데이터 조회 : SELECT

● 조건문 사용 : CASE

```
SELECT 고객아이디, 이름, 등급
      , CASE WHEN 등급='silver' THEN '일반고객'
              WHEN 등급='gold' THEN '우수고객'
              WHEN 등급='vip' THEN '최우수고객' END
FROM 고객;
```

```
SELECT 고객아이디, 이름, 등급
      , CASE 등급
        WHEN 'silver' THEN '일반고객'
        WHEN 'gold' THEN '우수고객'
        WHEN 'vip' THEN '최우수고객' END
FROM 고객;
```

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR	CASE WHEN WH
apple	정소화	gold	우수고객
banana	김선우	vip	최우수고객
carrot	고명석	gold	우수고객
kiwi	박서준	gold	우수고객
melon	성원용	gold	우수고객
orange	김용욱	silver	일반고객
peach	오형준	silver	일반고객
pear	채광주	silver	일반고객
plum	이하늘	vip	최우수고객

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR	CASE 등급 WHEN 'A' THEN '우수고객' WHEN 'B' THEN '최우수고객' WHEN 'C' THEN '일반고객' WHEN 'D' THEN '일반고객'
apple	정소화	gold	우수고객
banana	김선우	vip	최우수고객
carrot	고명석	gold	우수고객
kiwi	박서준	gold	우수고객
melon	성원용	gold	우수고객
orange	김용욱	silver	일반고객
peach	오형준	silver	일반고객
pear	채광주	silver	일반고객
plum	이하늘	vip	최우수고객

■ 연습문제

- 고객아이디와 아이디 길이를 출력하고,
아이디 길이가 6자 이상이면 '긴아이디', 5자면 '보통',
4자 이하면 '짧음'으로 표시하세요.

고객아이디 VARCHAR	아이디길이 INT	길이분류 VARCHAR
apple	5	보통
banana	6	긴아이디
carrot	6	긴아이디
kiwi	4	짧음
melon	5	보통
orange	6	긴아이디
peach	5	보통
pear	4	짧음
plum	4	짧음

- 제품 테이블에서 제품명과 단가를 출력하되,
백원 단위에서 반올림한 단가로 출력하세요.

제품명 VARCHAR	단가 INT	반올림단가 BIGINT
그냥만두	4500	5000
매운짬면	5500	6000
콩떡파이	2600	3000
맛난초콜렛	2500	3000
얼큰라면	1200	1000
통통우동	1550	2000
달콤비스킷	1500	2000

■ 연습문제

- 고객아이디와 아이디 길이를 출력하고,
아이디 길이가 6자 이상이면 '긴아이디', 5자면 '보통',
4자 이하면 '짧음'으로 표시하세요.

```
SELECT 이름,  
       LENGTH(이름) AS '이름길이',  
       IF(LENGTH(이름) >= 3, '긴이름', '짧은이름') AS '이름구분'  
FROM 고객;
```

- 제품 테이블에서 제품명과 단가를 출력하되,
백원 단위에서 반올림한 단가로 출력하세요.

```
SELECT 제품명, 단가,  
       ROUND(단가, -3) AS '반올림단가'  
FROM 제품;
```


■ 데이터 조회 : SELECT

● 집계 함수를 이용한 검색

- 통계적으로 계산한 결과를 검색
- 개수, 합계, 평균, 최대, 최소값 계산 기능 제공
- 속성값이 NULL 인 경우 제외
- SELECT 절 또는 HAVING 절 에서만 사용 가능 (WHERE 절 사용 불가)

함수	의미	사용가능 타입
COUNT	전체 개수	전체
MAX	최대값	전체
MIN	최소값	전체
SUM	합계	숫자
AVG	평균	숫자

■ 데이터 조회 : SELECT

- '제품' 테이블에서 모든 제품의 단가 평균을 검색

결과 #1 (1×1)
AVG(단가)
2,764.2857

```
SELECT AVG(단가)
FROM 제품;
```

- '제품' 테이블에서 '한빛제과'가 제조한 제품의 재고량 합계를 검색

결과 #1 (1×1)
SUM(재고량)
6,500

```
SELECT SUM(재고량)
FROM 제품
WHERE 제조업체='한빛제과';
```

■ 데이터 조회 : SELECT

● '고객' 테이블에서 고객의 수 검색

– 고객아이디 이용

결과 #1 (1×1)
전체고객수
7

```
SELECT COUNT(고객아이디) AS 전체고객수  
FROM 고객;
```

– 나이 이용

결과 #1 (1×1)
전체고객수
6

```
SELECT COUNT(나이) AS 전체고객수  
FROM 고객;
```

– * 이용

결과 #1 (1×1)
전체고객수
7

```
SELECT COUNT(*) AS 전체고객수  
FROM 고객;
```

■ 데이터 조회 : SELECT

- '제품' 테이블에서 제조업체의 수를 중복없이 검색

결과 #1 (1×1)	
제조업체 수	3

```
SELECT COUNT(DISTINCT 제조업체) AS '제조업체 수'  
FROM 제품;
```

- '고객' 테이블에서 고객직업의 수를 중복없이 검색

결과 #1 (1×1)	
고객직업 수	5

```
SELECT COUNT(DISTINCT 직업) AS '고객직업 수'  
FROM 고객;
```

■ 연습문제

- 제품 테이블에서 재고량이 가장 많은 값을 출력하세요.

최대재고 INT
5000

- 고객 테이블에서 적립금의 최대값과 최소값을 각각 출력하세요.

최대적립금 INT	최소적립금 INT
8000	0

- 제품 테이블에서 모든 제품 단가의 합계를 출력하세요.

단가합계 DECIMAL
19350

■ 연습문제

- 제품 테이블에서 재고량이 가장 많은 값을 출력하세요.

```
SELECT MAX(재고량) AS 최대재고  
FROM 제품;
```

- 고객 테이블에서 적립금의 최대값과 최소값을 각각 출력하세요.

```
SELECT MAX(적립금) AS 최대적립금,  
       MIN(적립금) AS 최소적립금  
FROM 고객;
```

- 제품 테이블에서 모든 제품 단가의 합계를 출력하세요.

```
SELECT SUM(단가) AS 단가합계  
FROM 제품;
```

■ 데이터 조회 : SELECT

● 그룹별 검색 : GROUP BY

```
SELECT [DISTINCT] 속성 리스트  
FROM 테이블 리스트  
[WHERE 조건]  
[GROUP BY 속성 리스트 [HAVING 조건]]  
[ORDER BY 속성 리스트 [ASC | DESC]]
```

- 특정 속성의 값이 같은 튜플을 모아 그룹을 만들어 그룹별 검색 가능
- SELECT 절에는 그룹으로 묶은 속성과 집계 함수만 사용 가능
- HAVING 키워드를 이용하여 그룹에 대한 조건 작성 가능

■ 데이터 조회 : SELECT

- '주문' 테이블에서 주문제품별 수량의 합계 검색

주문 (2x5)	
주문제품	총주문수량
p01	24
p02	58
p03	52
p04	15
p06	81

```
SELECT 주문제품, SUM(수량) AS 총주문수량  
FROM 주문  
GROUP BY 주문제품;
```


■ 데이터 조회 : SELECT

● 그룹별 검색 : GROUP BY

주문제품	수량
p03	10
p01	5
p06	45
p02	8
p06	36
p01	19
p03	22
p02	50
p04	15
p03	20

	주문제품	총주문수량
1	p01	24
2	p02	58
3	p03	52
4	p04	15
5	p06	81

- 주문제품이 같은 튜플을 모아 그룹으로 만들고, 그룹별로 수량의 합계 계산

■ 데이터 조회 : SELECT

- '제품' 테이블에서 제조업체별로 제조한 제품의 개수와 제품 중 최고가를 검색

제품 (3×3)		
제조업체	제품수	최고가
대한식품	2	4,500
민국푸드	2	5,500
한빛제과	3	2,600

```
SELECT 제조업체, COUNT(*) AS 제품수, MAX(단가) AS 최고가
FROM 제품
GROUP BY 제조업체;
```

■ 데이터 조회 : SELECT

- '제품' 테이블에서 제품을 3개 이상 제조한 제조업체별로 제품의 개수와 제품 중 가장 비싼 단가를 검색

제품 (3×1)		
제조업체	제품수	최고가
한빛제과	3	2,600

```
SELECT 제조업체, COUNT(*) AS 제품수, MAX(단가) AS 최고가
FROM 제품
GROUP BY 제조업체
HAVING COUNT(*) >= 3;
```

■ 데이터 조회 : SELECT

- '고객' 테이블에서 적립금 평균이 1000원 이상인 등급에 대해 등급별 고객 수와 적립금 평균을 검색

고객 (3x2)		
등급	고객수	평균적립금
gold	3	3,500.0000
vip	1	2,500.0000

```
SELECT 등급, COUNT(*) AS 고객수, AVG(적립금) AS 평균적립금
FROM 고객
GROUP BY 등급
HAVING AVG(적립금) >= 1000;
```

■ 데이터 조회 : SELECT

- '주문' 테이블에서 각 주문고객이 주문한 제품의 총주문수량을 주문제품별로 검색

주문 (3×9)		
주문제품	주문고객	총주문수량
p01	banana	19
p01	melon	5
p02	carrot	8
p02	pear	50
p03	apple	32
p03	carrot	20
p04	banana	15
p06	banana	45
p06	melon	36

```
SELECT 주문제품, 주문고객, SUM(수량) AS 총주문수량
FROM 주문
GROUP BY 주문제품, 주문고객;
```

■ 연습문제

- 등급별 고객 수를 구하고,
인원수가 많은 순서로 정렬하세요.

등급 VARCHAR	인원수 BIGINT
gold	4
silver	3
vip	2

- 직업이 없는(NULL) 고객은 '미기재'로 묶어서
직업별 인원수를 구하고
인원수 내림차순으로 정렬하세요

직업 VARCHAR	인원수 BIGINT
회사원	2
학생	2
의사	1
간호사	1
가수	1
교사	1
배우	1

- 고객아이디의 첫 글자별 인원수를 구하세요.

아이디첫글자 VARCHAR	인원수 BIGINT
a	1
b	1
c	1
k	1
m	1
o	1
p	3

■ 연습문제

- 등급별 고객 수를 구하고 인원수가 많은 순서로 정렬하세요.

```
SELECT 등급, COUNT(*) AS 인원수  
FROM 고객  
GROUP BY 등급  
ORDER BY 인원수 DESC;
```

- 직업이 없는(NULL) 고객은 '미기재'로 묶어서 직업별 인원수를 구하고
인원수 내림차순으로 정렬하세요

```
SELECT CASE WHEN 직업 IS NULL THEN '미기재'  
           ELSE 직업 END AS 직업,  
       COUNT(*) AS 인원수  
FROM 고객  
GROUP BY CASE WHEN 직업 IS NULL THEN '미기재' ELSE 직업 END  
ORDER BY 인원수 DESC;
```

- 고객아이디의 첫 글자별 인원수를 구하세요.

```
SELECT SUBSTR(고객아이디, 1, 1) AS 아이디첫글자,  
       COUNT(*) AS 인원수  
FROM 고객  
GROUP BY SUBSTR(고객아이디, 1, 1)  
ORDER BY 아이디첫글자;
```

■ 연습문제

- 주문고객별 주문 건수와
총 수량을 구하세요.

주문고객 VARCHAR	주문건수 BIGINT	총수량 DECIMAL
banana	3	79
melon	2	41
carrot	2	28
apple	2	32
pear	1	50

- 등급별로 '성인(나이 $\geq$ 20)' 수와
'미성년(나이 $<$ 20)' 수를 구하세요.

등급 VARCHAR	성인수 DECIMAL	미성년수 DECIMAL
gold	4	0
silver	2	0
vip	2	0

■ 연습문제

- 주문고객별 주문 건수와 총 수량을 구하세요.

```
SELECT 주문고객, COUNT(*) AS 주문건수, SUM(수량) AS 총수량  
FROM 주문  
GROUP BY 주문고객  
ORDER BY 주문건수 DESC;
```

- 등급별로 '성인(나이≥20)' 수와 '미성년(나이<20)' 수를 구하세요.

```
SELECT 등급,  
       SUM(CASE WHEN 나이 >= 20 THEN 1 ELSE 0 END) AS 성인수,  
       SUM(CASE WHEN 나이 < 20 THEN 1 ELSE 0 END) AS 미성년수  
FROM 고객  
GROUP BY 등급  
ORDER BY 등급;
```

■ 데이터 조회 : SELECT

● 여러 테이블에 대한 조인 검색

- 조인 : 여러 개의 테이블을 연결하여 하나의 테이블처럼 사용하는 것
- 조인 속성 : 조인 검색을 위해 테이블 간의 연결되는 속성
- 연결하려는 테이블 간의 조인 속성은 달라도 되지만 도메인은 같아야 됨
- 일반적으로 외래키가 조인 속성으로 사용
- 같은 이름의 속성이 서로 다른 테이블에 존재할 수 있기 때문에
속성 이름 앞에 해당 속성이 소속된 테이블의 이름 또는 별칭을 표시

구식 조인 (old-style join)
비ANSI 조인 (non-ANSI join)

```
SELECT 속성  
FROM 테이블1, 테이블2  
WHERE 테이블1.속성 = 테이블2.속성;
```

명시적 조인 (explicit join)
ANSI 표준 조인 (ANSI join)

```
SELECT 속성  
FROM 테이블1  
JOIN 테이블2  
ON 테이블1.속성 = 테이블2.속성  
JOIN 테이블3  
ON 테이블1.속성 = 테이블3.속성;
```

■ 데이터 조회 : SELECT

● 여러 테이블에 대한 조인 검색



■ 데이터 조회 : SELECT

● '주문' 테이블과 '고객' 테이블 조인

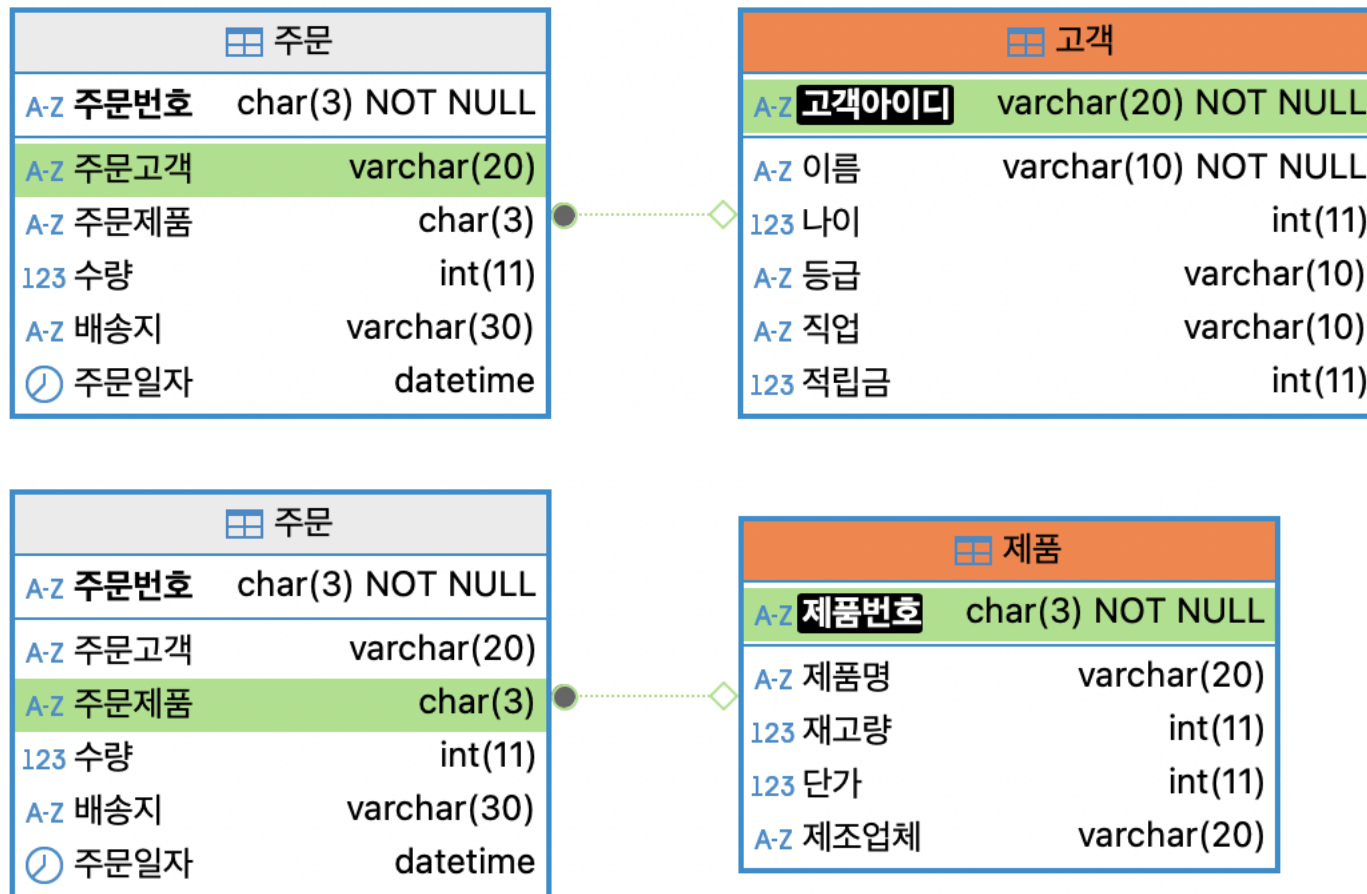
주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
apple	정소화	20	gold	학생	1000
banana	김선우	25	vip	간호사	2500
carrot	고명석	28	gold	교사	4500
kiwi	박서준	29	gold	배우	4000
melon	성원용	35	gold	회사원	5000
orange	김용욱	22	silver	학생	0
peach	오형준	NULL	silver	의사	300
pear	채광주	31	silver	회사원	500
plum	이하늘	40	vip	가수	8000

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME	고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00	apple	정소화	20	gold	학생	1000
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00	melon	성원용	35	gold	회사원	5000
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00	banana	김선우	25	vip	간호사	2500
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00	carrot	고명석	28	gold	교사	4500
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00	melon	성원용	35	gold	회사원	5000
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00	banana	김선우	25	vip	간호사	2500
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00	apple	정소화	20	gold	학생	1000
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00	pear	채광주	31	silver	회사원	500
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00	banana	김선우	25	vip	간호사	2500
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00	carrot	고명석	28	gold	교사	4500

■ 데이터 조회 : SELECT

● ERD (Entity Relationship Diagram)



- 두 테이블의 "공통 속성값"을 기준으로 데이터를 결합

■ 데이터 조회 : SELECT

- 'banana' 고객이 주문한 제품의 제품명을 검색



주문고객 VARCHAR	주문제품 CHAR	제품번호 CHAR	제품명 VARCHAR
banana	p01	p01	그냥만두
banana	p04	p04	맛난초콜렛
banana	p06	p06	통통우동

```
SELECT O.주문고객, O.주문제품, P.제품번호, P.제품명
FROM 제품 AS P
JOIN 주문 AS O
ON O.주문제품 = P.제품번호
WHERE O.주문고객 = 'banana';
```

■ 데이터 조회 : SELECT

- 나이가 30세 이상의 고객이 주문한 제품의 주문제품과 주문일자 검색

주문제품 CHAR	주문일자 DATETIME	주문고객 VARCHAR	고객아이디 VARCHAR	나이 INT
p01	2013-01-10 00:00:00	melon	plum	40
p06	2013-02-20 00:00:00	melon	melon	35
p02	2013-04-10 00:00:00	pear	pear	31

주문제품 CHAR	주문일자 DATETIME
p01	2013-01-10 00:00:00
p06	2013-02-20 00:00:00
p02	2013-04-10 00:00:00

```
SELECT 주문.주문제품, 주문.주문일자  
FROM 고객  
JOIN 주문  
ON 고객.고객아이디 = 주문.주문고객  
WHERE 고객.나이 >= 30;
```

■ 데이터 조회 : SELECT

● '고명석' 고객이 주문한 제품의 제품명 검색

이름 VARCHAR	주문제품 CHAR	주문일자 DATETIME	제품명 VARCHAR
고명석	p02	2013-02-01 00:00:00	매운짬면
고명석	p03	2013-05-22 00:00:00	콩떡파이

```
SELECT C.이름, O.주문제품, O.주문일자, P.제품명  
FROM 고객 AS C  
JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객  
JOIN 제품 AS P  
ON O.주문제품 = P.제품번호  
WHERE C.이름 = '고명석';
```


■ 데이터 조회 : SELECT

- '서울시 마포구'를 배송지로 등록한 적이 있는 고객의 이름, 직업, 주문한 제품명, 단가, 제조업체 검색

이름 VARCHAR	직업 VARCHAR	제품명 VARCHAR	단가 INT	제조업체 VARCHAR
정소화	학생	쿵떡파이	2600	한빛제과

```
SELECT C.이름, C.직업, P.제품명, P.단가, P.제조업체  
FROM 고객 AS C  
JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객  
JOIN 제품 AS P  
ON O.주문제품 = P.제품번호  
WHERE O.배송지 = '서울시 마포구';
```

■ 데이터 조회 : SELECT

- '그냥만두'를 주문한 적이 있는 고객의 이름과 나이, 수량 검색

이름 VARCHAR	나이 INT	수량 INT
성원용	35	5
김선우	25	19

```
SELECT C.이름, C.나이, O.수량  
FROM 고객 AS C  
JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객  
JOIN 제품 AS P  
ON O.주문제품 = P.제품번호  
AND P.제품명 = '그냥만두';
```

■ 연습문제

● 단가가 5,000원 이상인 제품의

제품명과 단가, 주문번호, 주문일자, 주문고객을 조회하세요.

제품명 VARCHAR	단가 INT	주문번호 CHAR	주문일자 DATETIME	주문고객 VARCHAR
매운쫄면	5500	o04	2013-02-01 00:00:00	carrot
매운쫄면	5500	o08	2013-04-10 00:00:00	pear

● 나이가 25세 미만인 고객의

이름과 주문제품, 주문일자, 수량, 배송지를 조회하세요.

이름 VARCHAR	주문제품 CHAR	주문일자 DATETIME	수량 INT	배송지 VARCHAR
정소화	p03	2013-01-01 00:00:00	10	서울시 마포구
정소화	p03	2013-03-15 00:00:00	22	서울시 영등포구

■ 연습문제

● 단가가 5,000원 이상인 제품의

제품명과 단가, 주문번호, 주문일자, 주문고객을 조회하세요.

```
SELECT P.제품명, P.단가, O.주문번호, O.주문일자, O.주문고객  
FROM 제품 AS P  
JOIN 주문 AS O  
ON P.제품번호 = O.주문제품  
WHERE P.단가 >= 5000;
```

● 나이가 25세 미만인 고객의

이름과 주문제품, 주문일자, 수량, 배송지를 조회하세요.

```
SELECT C.이름, O.주문제품, O.주문일자, O.수량, O.배송지  
FROM 고객 AS C  
JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객  
WHERE C.나이 < 25;
```

■ 데이터 조회 : SELECT

● 조인의 종류

– Outer Join

- 연결되는 컬럼 값이 없더라도 데이터를 검색해주는 조인
- 기준이 되는 테이블을 Outer, 반대쪽을 Inner 라고 표현
- Left Outer 조인과 Right Outer 조인으로 구분

(DB의 종류에 따라 Full Outer 조인도 사용)

```
SELECT 속성 리스트  
FROM 테이블1  
LEFT JOIN 테이블2  
ON 테이블1.속성 = 테이블2.속성
```

Join

ID	P_NAME	CODE
1	G5	10
2	GALAXY S7	20
3	MI5	30

CODE	M_NAME
10	LG
20	SAMSUNG
40	SONY



ID	P_NAME	CODE	M_NAME
1	G5	10	LG
2	GALAXY S7	20	SAMSUNG

Outer Join

ID	P_NAME	CODE
1	G5	10
2	GALAXY S7	20
3	MI5	30

CODE	M_NAME
10	LG
20	SAMSUNG
40	SONY



ID	P_NAME	CODE	M_NAME
1	G5	10	LG
2	GALAXY S7	20	SAMSUNG
3	MI5	30	(NULL)

■ 데이터 조회 : SELECT

● 고객 / 주문 테이블 Equi 조인

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
apple	정소화	20	gold	학생	1000
banana	김선우	25	vip	간호사	2500
carrot	고명석	28	gold	교사	4500
kiwi	박서준	29	gold	배우	4000
melon	성원용	35	gold	회사원	5000
orange	김용욱	22	silver	학생	0
peach	오형준	NULL	silver	의사	300
pear	채광주	31	silver	회사원	500
plum	이하늘	40	vip	가수	8000

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00

□ : 연결되는 데이터 없음



이름 VARCHAR	나이 INT	고객아이디 VARCHAR	주문고객 VARCHAR	주문제품 CHAR	배송지 VARCHAR
정소화	20	apple	apple	p03	서울시 마포구
정소화	20	apple	apple	p03	서울시 영등포구
김선우	25	banana	banana	p06	경기도 부천시
김선우	25	banana	banana	p01	충청북도 보은군
김선우	25	banana	banana	p04	전라남도 목포시
고명석	28	carrot	carrot	p02	부산시 금정구
고명석	28	carrot	carrot	p03	경기도 안양시
성원용	35	melon	melon	p01	인천시 계양구
성원용	35	melon	melon	p06	경기도 용인시
채광주	31	pear	pear	p02	강원도 춘천시

SELECT 이름, 나이, 고객아이디
 , 주문고객, 주문제품, 배송지
 FROM 고객
 JOIN 주문
 ON 고객.고객아이디 = 주문.주문고객
 ORDER BY 고객아이디;

■ 데이터 조회 : SELECT

● 고객 / 주문 테이블 (Left) Outer 조인

고객아이디 VARCHAR	이름 VARCHAR	나이 INT	등급 VARCHAR	직업 VARCHAR	적립금 INT
apple	정소화	20	gold	학생	1000
banana	김선우	25	vip	간호사	2500
carrot	고명석	28	gold	교사	4500
kiwi	박서준	29	gold	배우	4000
melon	성원웅	35	gold	회사원	5000
orange	김용욱	22	silver	학생	0
peach	오형준	NULL	silver	의사	300
pear	채광주	31	silver	회사원	500
plum	이하늘	40	vip	가수	8000

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00

□ : 연결되는 데이터가 없어도 조인



이름 VARCHAR	나이 INT	고객아이디 VARCHAR	주문고객 VARCHAR	주문제품 CHAR	배송지 VARCHAR
정소화	20	apple	apple	p03	서울시 마포구
정소화	20	apple	apple	p03	서울시 영등포구
김선우	25	banana	banana	p06	경기도 부천시
김선우	25	banana	banana	p01	충청북도 보은군
김선우	25	banana	banana	p04	전라남도 목포시
고명석	28	carrot	carrot	p02	부산시 금정구
고명석	28	carrot	carrot	p03	경기도 안양시
박서준	29	kiwi	NULL	NULL	NULL
성원웅	35	melon	melon	p01	인천시 계양구
성원웅	35	melon	melon	p06	경기도 용인시
김용욱	22	orange	NULL	NULL	NULL
오형준	NULL	peach	NULL	NULL	NULL
채광주	31	pear	pear	p02	강원도 춘천시
이하늘	40	plum	NULL	NULL	NULL

SELECT 이름, 나이, 고객아이디
 , 주문고객, 주문제품, 배송지
 FROM 고객
 LEFT JOIN 주문
 ON 고객.고객아이디 = 주문.주문고객
 ORDER BY 고객아이디;

□ : 연결되는 데이터가 없어서 NULL로 표현

■ 데이터 조회 : SELECT

● 제품 / 주문 테이블 Equi 조인

제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운쫄면	2500	5500	민국푸드
p03	콩떡파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00

□ : 연결되는 데이터 없음



제품명 VARCHAR	단가 INT	제품번호 CHAR	주문제품 CHAR	주문고객 VARCHAR	배송지 VARCHAR
그냥만두	4500	p01	p01	melon	인천시 계양구
그냥만두	4500	p01	p01	banana	충청북도 보은군
매운쫄면	5500	p02	p02	carrot	부산시 금정구
매운쫄면	5500	p02	p02	pear	강원도 춘천시
콩떡파이	2600	p03	p03	apple	서울시 마포구
콩떡파이	2600	p03	p03	carrot	경기도 안양시
콩떡파이	2600	p03	p03	apple	서울시 영등포구
맛난초콜렛	2500	p04	p04	banana	전라남도 목포시
통통우동	1550	p06	p06	melon	경기도 용인시
통통우동	1550	p06	p06	banana	경기도 부천시

SELECT 제품명, 단가, 제품번호
 , 주문제품, 주문고객, 배송지
 FROM 제품
 JOIN 주문
 ON 제품.제품번호 = 주문.주문제품
 ORDER BY 제품번호;

■ 데이터 조회 : SELECT

● 제품 / 주문 테이블 (Left) Outer 조인

제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운쫄면	2500	5500	민국푸드
p03	콩떡파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VARCHAR	주문일자 DATETIME
o06	banana	p01	19	충청북도 보은군	2013-03-02 00:00:00
o02	melon	p01	5	인천시 계양구	2013-01-10 00:00:00
o08	pear	p02	50	강원도 춘천시	2013-04-10 00:00:00
o04	carrot	p02	8	부산시 금정구	2013-02-01 00:00:00
o07	apple	p03	22	서울시 영등포구	2013-03-15 00:00:00
o01	apple	p03	10	서울시 마포구	2013-01-01 00:00:00
o10	carrot	p03	20	경기도 안양시	2013-05-22 00:00:00
o09	banana	p04	15	전라남도 목포시	2013-04-11 00:00:00
o05	melon	p06	36	경기도 용인시	2013-02-20 00:00:00
o03	banana	p06	45	경기도 부천시	2013-01-11 00:00:00

□ : 연결되는 데이터가 없어도 조인



제품명 VARCHAR	단가 INT	제품번호 CHAR	주문제품 CHAR	주문고객 VARCHAR	배송지 VARCHAR
그냥만두	4500	p01	p01	melon	인천시 계양구
그냥만두	4500	p01	p01	banana	충청북도 보은군
매운쫄면	5500	p02	p02	carrot	부산시 금정구
매운쫄면	5500	p02	p02	pear	강원도 춘천시
콩떡파이	2600	p03	p03	apple	서울시 마포구
콩떡파이	2600	p03	p03	apple	서울시 영등포구
콩떡파이	2600	p03	p03	carrot	경기도 안양시
맛난초콜렛	2500	p04	p04	banana	전라남도 목포시
얼큰라면	1200	p05	NULL	NULL	NULL
통통우동	1550	p06	p06	banana	경기도 부천시
통통우동	1550	p06	p06	melon	경기도 용인시
달콤비스킷	1500	p07	NULL	NULL	NULL

SELECT 제품명, 단가, 제품번호
 , 주문제품, 주문고객, 배송지
 FROM 제품
 LEFT JOIN 주문
 ON 제품.제품번호 = 주문.주문제품
 ORDER BY 제품번호;

□ : 연결되는 데이터가 없어서 NULL로 표현

■ 연습문제

- 주문이 없는 고객을 포함해서
모든 고객의 고객아이디, 이름, 주문번호를 출력하세요.

- 주문이 없는 제품의
제품번호, 제품명, 주문번호를 출력하세요.

제품번호 CHAR	제품명 VARCHAR	주문번호 CHAR
p05	얼큰라면	NULL
p07	달콤비스킷	NULL

고객아이디 VARCHAR	이름 VARCHAR	주문번호 CHAR
apple	정소화	o01
apple	정소화	o07
banana	김선우	o03
banana	김선우	o06
banana	김선우	o09
carrot	고명석	o04
carrot	고명석	o10
kiwi	박서준	NULL
melon	성원용	o02
melon	성원용	o05
orange	김용욱	NULL
peach	오형준	NULL
pear	채광주	o08
plum	이하늘	NULL

- 주문이 없거나 주문 수량이 5개 이하인 고객의
고객아이디, 이름, 주문번호, 수량을 출력하세요.

고객아이디 VARCHAR	이름 VARCHAR	주문번호 CHAR	수량 INT
kiwi	박서준	NULL	NULL
melon	성원용	o02	5
orange	김용욱	NULL	NULL
peach	오형준	NULL	NULL
plum	이하늘	NULL	NULL

■ 연습문제

● 주문이 없는 고객을 포함해서

모든 고객의 고객아이디, 이름, 주문번호를 출력하세요.

```
SELECT C.고객아이디, C.이름, O.주문번호  
FROM 고객 AS C  
LEFT JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객;
```

● 주문이 없는 제품의 제품번호, 제품명, 주문번호를 출력하세요.

```
SELECT P.제품번호, P.제품명, O.주문번호  
FROM 제품 AS P  
LEFT JOIN 주문 AS O  
ON P.제품번호 = O.주문제품  
WHERE O.주문번호 IS NULL;
```

● 주문이 없거나 주문 수량이 5개 이하인 고객의

고객아이디, 이름, 주문번호, 수량을 출력하세요.

```
SELECT C.고객아이디, C.이름, O.주문번호, O.수량  
FROM 고객 AS C  
LEFT JOIN 주문 AS O  
ON C.고객아이디 = O.주문고객  
WHERE O.수량 <= 5 OR O.주문번호 IS NULL;
```

■ 데이터 조회 : SELECT

● 부속 질의문(서브쿼리)을 이용한 검색

- SELECT 문 안에 또 다른 SELECT 문을 포함하는 질의
- 하나의 튜플(행)을 반환하는 단일 행 부속 질의문과
여러 개의 튜플(행)을 반환하는 다중 행 부속 질의문 사용 가능
- 부속 질의문이 먼저 실행되고, 그 결과를 이용하여 상위 질의문이 실행
- 부속 질의문은 SELECT / FROM / WHERE / HAVING절 등 어디든 사용 가능
 - 괄호로 묶어서 작성해야 하며, ORDER BY 절을 사용할 수 없음

```
SELECT 속성 리스트  
FROM 테이블  
WHERE 테이블.컬럼 = (SELECT 속성  
                      FROM 테이블  
                      WHERE 테이블.컬럼 = 값);
```

■ 데이터 조회 : SELECT

● 부속 질의문(서브쿼리)을 이용한 검색

- 단일 행 부속 질의문은 = 등의 비교 연산자를 사용하면 되지만
다중 행 부속 질의문은 별도의 연산자를 사용해야 실행 가능

연산자	설명	사용 예
IN	부속 질의문의 결과 값 중 일치하는 것이 있으면 참 일치하는 튜플만 조회	직업이 회사원인 고객들이 주문한 일자
NOT IN	부속 질의문의 결과 값 중 일치하는 것이 없으면 참 일치하는 튜플만 조회	배송지가 서울이 아닌 고객들의 나이

■ 데이터 조회 : SELECT

● 단일 행 서브쿼리 (Single-row Subquery)

- 1개의 결과만 반환
- 비교 연산자 (=, >, <, >=, <=, <>, !=) 와 함께 사용 가능
- 적립금이 가장 많은 고객 찾기

```
SELECT MAX(적립금)  
FROM 고객;
```

MAX(적립금) INT
8000

```
SELECT 이름, 나이, 직업, 적립금  
FROM 고객  
WHERE 적립금 = 8000;
```

이름 VARCHAR	나이 INT	직업 VARCHAR	적립금 INT
이하늘	40	가수	8000



```
SELECT 이름, 나이, 직업, 적립금  
FROM 고객  
WHERE 적립금 =(SELECT MAX(적립금)  
FROM 고객);
```

■ 데이터 조회 : SELECT

● 단일 행 서브쿼리 (Single-row Subquery)

- 평균 단가보다 비싼 제품 찾기

```
SELECT AVG(단가)  
FROM 제품;
```

AVG(단가) DECIMAL
2764.2857

```
SELECT 제품명, 단가  
FROM 제품  
WHERE 단가 > 2764.2857;
```

제품명 VARCHAR	단가 INT
그냥만두	4500
매운짬면	5500



```
SELECT 제품명, 단가  
FROM 제품  
WHERE 단가 > (SELECT AVG(단가)  
FROM 제품);
```

■ 데이터 조회 : SELECT

● 단일 행 서브쿼리 (Single-row Subquery)

- '달콤비스켓' 을 만든 제조업체에서 제조한 제품 찾기

```
SELECT 제조업체  
FROM 제품  
WHERE 제품명 = '달콤비스켓';
```

제조업체 VARCHAR
한빛제과

```
SELECT 제품명, 단가  
FROM 제품  
WHERE 제조업체 = '한빛제과';
```

제품명 VARCHAR	단가 INT
쿵떡파이	2600
맛난초콜렛	2500
달콤비스켓	1500



```
SELECT 제품명, 단가  
FROM 제품  
WHERE 제조업체 = (SELECT 제조업체  
FROM 제품  
WHERE 제품명 = '달콤비스켓');
```


■ 데이터 조회 : SELECT

● 다중 행 서브쿼리 (Multi-row Subquery)

- 여러 개의 결과를 반환
- IN, NOT IN 과 같은 다중 행 연산자 필요
- 특정 고객들이 주문한 제품 찾기 (IN)

```
SELECT 고객아이디  
FROM 고객  
WHERE 등급 = 'gold';
```

고객아이디 VARCHAR
apple
carrot
kiwi
melon

```
SELECT 주문번호, 주문제품  
FROM 주문  
WHERE 주문고객 IN ('apple', 'carrot', 'kiwi', 'melon');
```

주문번호 CHAR	주문제품 CHAR
o01	p03
o02	p01
o04	p02
o05	p06
o07	p03
o10	p03



```
SELECT 주문번호, 주문제품  
FROM 주문  
WHERE 주문고객 IN (SELECT 고객아이디  
                     FROM 고객  
                     WHERE 등급 = 'gold');
```

■ 데이터 조회 : SELECT

● 다중 행 서브쿼리 (Multi-row Subquery)

– banana 고객이 주문하지 않은 제품 찾기 (NOT IN)

```
SELECT 주문제품  
FROM 주문  
WHERE 주문고객 = 'banana';
```

주문제품 CHAR
p06
p01
p04

```
SELECT 제품명, 제조업체  
FROM 제품  
WHERE 제품번호 NOT IN ('p01', 'p04', 'p06');
```

제품명 VARCHAR	제조업체 VARCHAR
매운짬면	민국푸드
쿵떡파이	한빛제과
얼큰라면	대한식품
달콤비스킷	한빛제과



```
SELECT 제품명, 제조업체  
FROM 제품  
WHERE 제품번호 NOT IN (SELECT 주문제품  
                        FROM 주문  
                        WHERE 주문고객 = 'banana');
```

■ 데이터 조회 : SELECT

● 서브쿼리를 2번 이상 사용해야 하는 경우

- 특정 고객이 주문한 제품의 최고 단가보다 비싼 제품 찾기

```
SELECT 제품명, 단가
FROM 제품
WHERE 단가 > (
    SELECT MAX(단가)
    FROM 제품
    WHERE 제품번호 IN (
        SELECT 주문제품
        FROM 주문
        WHERE 주문고객 = 'banana'
    )
);
```



제품명 VARCHAR	단가 INT
매운짬면	5500

■ 데이터 조회 : SELECT

● 서브쿼리를 2번 이상 사용해야 하는 경우

- 모든 고객의 평균 주문 수량보다 많이 주문한 고객 찾기

```
SELECT 이름  
      , (SELECT AVG(수량) FROM 주문) AS "평균"  
FROM 고객  
WHERE 고객아이디 IN (  
  SELECT 주문고객  
  FROM 주문  
  GROUP BY 주문고객  
  HAVING SUM(수량) > (  
    SELECT AVG(수량)  
    FROM 주문  
  )  
);
```



이름 VARCHAR	평균 DECIMAL
정소화	23.0000
김선우	23.0000
고명석	23.0000
성원용	23.0000
채광주	23.0000

■ 데이터 조회 : SELECT

● 서브쿼리를 2번 이상 사용해야 하는 경우

- 나이가 가장 어린 고객이 주문한 제품의 제조업체 찾기

```
SELECT DISTINCT 제조업체  
FROM 제품  
WHERE 제품번호 IN (  
    SELECT 주문제품  
    FROM 주문  
    WHERE 주문고객 = (  
        SELECT 고객아이디  
        FROM 고객  
        WHERE 나이 = (  
            SELECT MIN(나이)  
            FROM 고객  
        )  
    )  
);
```



제조업체 VARCHAR
한빛제과

■ 데이터 조회 : SELECT

● 서브쿼리를 2번 이상 사용해야 하는 경우

- 한 번도 주문하지 않았고 평균 나이보다 많은 고객 찾기

```
SELECT 이름, 나이  
FROM 고객  
WHERE 나이 > (SELECT AVG(나이) FROM 고객)  
AND 고객아이디 NOT IN (  
    SELECT 주문고객  
    FROM 주문  
);
```



이름 VARCHAR	나이 INT
박서준	29
이하늘	40

■ 연습문제

- '한빛제과' 제품들의 평균 단가보다 비싼 제품의 제품명, 단가를 조회하세요.

제품명 VARCHAR	단가 INT
그냥만두	4500
매운짬면	5500
쿵떡파이	2600
맛난초콜렛	2500

- 가장 최근 주문한 고객이 주문한 모든 제품의 제조업체를 조회하세요.

제조업체 VARCHAR
민국푸드
한빛제과

■ 연습문제

- '한빛제과' 제품들의 평균 단가보다 비싼 제품의
제품명, 단가를 조회하세요.

```
SELECT 제품명, 단가
FROM 제품
WHERE 단가 > (
    SELECT AVG(단가)
    FROM 제품
    WHERE 제조업체 = '한빛제과'
);
```

- 가장 최근 주문한 고객이 주문한 모든 제품의 제조업체를 조회하세요.

```
SELECT DISTINCT p.제조업체
FROM 제품 p
WHERE p.제품번호 IN (
    SELECT o.주문제품
    FROM 주문 o
    WHERE o.주문고객 = (
        SELECT 주문고객
        FROM 주문
        WHERE 주문일자 = (
            SELECT MAX(주문일자) FROM 주문
        )
    )
);
```


■ 데이터 조회 : SELECT

● FROM 절 파생 테이블

- FROM 절에 오는 서브쿼리가 임시 테이블처럼 동작하는 형태
- 서브쿼리 결과를 테이블처럼 다시 SELECT/JOIN/WHERE로 다룰 수 있음

```
SELECT ...  
FROM (  
    SELECT ...  
    FROM ...  
    WHERE ...  
    GROUP BY ...  
) AS 별칭      -- ★ 별칭 필수  
WHERE ...;
```

- 파생 테이블 안에서 만들어진 컬럼은 밖에서 별칭으로 접근 가능

■ 데이터 조회 : SELECT

● FROM 절 파생 테이블

- 고객별 주문건수를 파생 테이블로 만들고 2건 이상만 이름과 함께 보기

고객아이디 VARCHAR	이름 VARCHAR	주문건수 BIGINT
banana	김선우	3
melon	성원용	2
apple	정소화	2
carrot	고명석	2

```
SELECT C.고객아이디, C.이름, T.주문건수
FROM 고객 AS C
JOIN (
  SELECT 주문고객, COUNT(*) AS 주문건수
  FROM 주문
  GROUP BY 주문고객
) AS T
ON T.주문고객 = C.고객아이디
WHERE T.주문건수 >= 2
ORDER BY T.주문건수 DESC;
```

■ 데이터 조회 : SELECT

● FROM 절 파생 테이블

- 업체별 평균단가를 만든 후 제품의 단가가 평균보다 비싼 제품만 보기

제품명 VARCHAR	단가 INT	제조업체 VARCHAR	평균단가 DECIMAL
그냥만두	4500	대한식품	2850.0000
매운짬면	5500	민국푸드	3525.0000
쿵떡파이	2600	한빛제과	2200.0000
맛난초콜렛	2500	한빛제과	2200.0000

```
SELECT P.제품명, P.단가, P.제조업체, A.평균단가
FROM 제품 AS P
JOIN (
    SELECT 제조업체, AVG(단가) AS 평균단가
    FROM 제품
    GROUP BY 제조업체
) AS A
ON A.제조업체 = P.제조업체
WHERE P.단가 > A.평균단가
ORDER BY P.제조업체, P.단가 DESC;
```

■ 데이터 조회 : SELECT

● 스칼라 서브쿼리 (Scalar Subquery)

- 단 하나의 값 (1개 열) 을 반환하는 서브쿼리

```
SELECT 컬럼1,  
      (SELECT 단일값 FROM ... WHERE ... ) AS 새컬럼  
FROM 테이블;
```

- 반드시 단일 값만 반환해야 됨
- 외부 테이블의 컬럼을 참조하는 것도 가능

■ 데이터 조회 : SELECT

● 스칼라 서브쿼리 (Scalar Subquery)

- 각 제품 옆에 자기 제조업체의 평균단가를 함께 보여주기

제품명 VARCHAR	단가 INT	제조업체 VARCHAR	평균단가 DECIMAL
그냥만두	4500	대한식품	2850.0000
매운짬면	5500	민국푸드	3525.0000
쿵떡파이	2600	한빛제과	2200.0000
맛난초콜렛	2500	한빛제과	2200.0000

```
SELECT 제품명,  
       단가,  
       제조업체,  
       (SELECT AVG(단가)  
        FROM 제품 AS X  
        WHERE X.제조업체 = 제품.제조업체) AS 제조사_평균단가  
FROM 제품  
ORDER BY 제조업체, 단가 DESC;
```

■ 데이터 조회 : SELECT

● 스칼라 서브쿼리 (Scalar Subquery)

- 각 고객의 정보와 최종 주문일자를 함께 보여주기

고객아이디 VARCHAR	이름 VARCHAR	최종주문일자 DATETIME
carrot	고명석	2013-05-22 00:00:00
banana	김선우	2013-04-11 00:00:00
pear	채광주	2013-04-10 00:00:00
apple	정소화	2013-03-15 00:00:00
melon	성원용	2013-02-20 00:00:00
kiwi	박서준	NULL
orange	김용욱	NULL
peach	오형준	NULL
plum	이하늘	NULL

```
SELECT C.고객아이디,  
       C.이름,  
       (SELECT MAX(주문일자)  
        FROM 주문 AS O  
        WHERE O.주문고객 = C.고객아이디) AS 최종주문일자  
FROM 고객 AS C  
ORDER BY 최종주문일자 DESC;
```

■ 데이터 수정 : UPDATE

UPDATE 테이블명

SET 속성 이름1 = 값1, 속성 이름2 = 값2, ..., 속성 이름n = 값n;
[WHERE 조건];

- SET 키워드 다음에 속성 값을 어떻게 수정할 것인지 지정
- WHERE 절을 생략하면 테이블에 존재하는 모든 튜플을 대상으로 수정

※ DB 접속 툴 HeidiSQL 사용 시 Auto Commit 비활성화

SET @@AUTOCOMMIT = 0;

■ 데이터 수정 : UPDATE

- '제품' 테이블에서 제품번호가 'p03'인 제품의 제품명을 '통큰파이'로 수정

제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운짬면	2500	5500	민국푸드
p03	쿵떡파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과



제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운짬면	2500	5500	민국푸드
p03	통큰파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과

```
UPDATE 제품  
SET 제품명 = '통큰파이'  
WHERE 제품번호 = 'p03';
```


■ 데이터 수정 : UPDATE

● 제품번호가 'p03'인 제품의 단가를 10% 인상

제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운짬면	2500	5500	민국푸드
p03	통큰파이	3600	2600	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과



제품번호 CHAR	제품명 VARCHAR	재고량 INT	단가 INT	제조업체 VARCHAR
p01	그냥만두	5000	4500	대한식품
p02	매운짬면	2500	5500	민국푸드
p03	통큰파이	3600	2860	한빛제과
p04	맛난초콜렛	1250	2500	한빛제과
p05	얼큰라면	2200	1200	대한식품
p06	통통우동	1000	1550	민국푸드
p07	달콤비스킷	1650	1500	한빛제과

```
UPDATE 제품
SET 단가 = 단가 * 1.1
WHERE 제품번호 = 'p03';
```

■ 데이터 수정 : UPDATE

- 주문내역 중 고객이름이 '정소화'인 고객의 수량을 5개로 수정

고객아이디 VARCHAR

apple

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VA
o01	apple	p03	10	서울시 마
o02	melon	p01	5	인천시 가
o03	banana	p06	45	경기도 북
o04	carrot	p02	8	부산시 금
o05	melon	p06	36	경기도 용
o06	banana	p01	19	충청북도
o07	apple	p03	22	서울시 양
o08	pear	p02	50	강원도 춘
o09	banana	p04	15	전라남도
o10	carrot	p03	20	경기도 양



주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	수량 INT	배송지 VA
o01	apple	p03	5	서울시 마
o02	melon	p01	5	인천시 가
o03	banana	p06	45	경기도 북
o04	carrot	p02	8	부산시 금
o05	melon	p06	36	경기도 용
o06	banana	p01	19	충청북도
o07	apple	p03	5	서울시 양
o08	pear	p02	50	강원도 춘
o09	banana	p04	15	전라남도
o10	carrot	p03	20	경기도 양

```
UPDATE 주문
SET 수량 = 5
WHERE 주문고객 IN (SELECT 고객아이디
                    FROM 고객
                    WHERE 고객이름 = '정소화');
```

■ 데이터 삭제 : DELETE

```
DELETE  
FROM 테이블명  
[WHERE 조건];
```

- DELETE 는 조건을 만족하는 행(row)을 삭제
- WHERE 절을 생략하면 테이블 전체 행이 삭제됨 (매우 주의!)
- 조건은 단순 비교, 범위 조건, 서브쿼리 등 다양하게 지정 가능

■ 데이터 삭제 : DELETE

● 주문일자가 2013년 5월 22일인 주문내역 삭제

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o01	apple	p03	2013-01-01 00
o02	melon	p01	2013-01-10 00
o03	banana	p06	2013-01-11 00:
o04	carrot	p02	2013-02-01 00
o05	melon	p06	2013-02-20 00
o06	banana	p01	2013-03-02 00
o07	apple	p03	2013-03-15 00
o08	pear	p02	2013-04-10 00
o09	banana	p04	2013-04-11 00
o10	carrot	p03	2013-05-22 00



주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o01	apple	p03	2013-01-01 00
o02	melon	p01	2013-01-10 00
o03	banana	p06	2013-01-11 00:
o04	carrot	p02	2013-02-01 00
o05	melon	p06	2013-02-20 00
o06	banana	p01	2013-03-02 00
o07	apple	p03	2013-03-15 00
o08	pear	p02	2013-04-10 00
o09	banana	p04	2013-04-11 00

```
DELETE FROM 주문  
WHERE 주문일자 = '2013-05-22';
```

■ 데이터 삭제 : DELETE

- 수량이 10 보다 적거나 제품명이 '통통우동' 인 주문내역 삭제

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o01	apple	p03	2013-01-01 00
o02	melon	p01	2013-01-10 00
o03	banana	p06	2013-01-11 00
o04	carrot	p02	2013-02-01 00
o05	melon	p06	2013-02-20 00
o06	banana	p01	2013-03-02 00
o07	apple	p03	2013-03-15 00
o08	pear	p02	2013-04-10 00
o09	banana	p04	2013-04-11 00



주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o06	banana	p01	2013-03-02 00
o08	pear	p02	2013-04-10 00
o09	banana	p04	2013-04-11 00

```
DELETE FROM 주문
WHERE 수량 < 10
OR 주문제품 = (SELECT 제품번호
                FROM 제품
                WHERE 제품명 = '통통우동');
```

■ 데이터 삭제 : DELETE

- 등급이 silver 이고 직업이 회사원인 고객의 주문내역 삭제

고객아이디 VARCHAR	이름 VARCHAR	등급 VARCHAR	직업 VARCHAR	나이 VARCHAR
apple	정소화	gold	학생	20
banana	김선우	vip	간호사	25
carrot	고명석	gold	교사	30
kiwi	박서준	gold	배우	35
melon	성원용	gold	회사원	40
orange	김용욱	silver	학생	20
peach	오형준	silver	의사	25
pear	채광주	silver	회사원	30
plum	이하늘	vip	가수	35

주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o06	banana	p01	2013-03-02 00:00:00
o08	pear	p02	2013-04-10 00:00:00
o09	banana	p04	2013-04-11 00:00:00



주문번호 CHAR	주문고객 VARCHAR	주문제품 CHAR	주문일자 DATETIME
o06	banana	p01	2013-03-02 00:00:00
o09	banana	p04	2013-04-11 00:00:00

```
DELETE FROM 주문
WHERE 주문고객 = (SELECT 고객아이디
                  FROM 고객
                  WHERE 등급 = 'silver'
                  AND 직업 = '회사원');
```